



UNIVERSITÀ DEGLI STUDI DI CATANIA
DIPARTIMENTO DI MATEMATICA E INFORMATICA
CORSO DI LAUREA MAGISTRALE IN INFORMATICA

Petronio Petronio Davide

Crittografia

APPUNTI LEZIONI

Prof. Catalano

Anno Accademico 2025 - 2026

Indice

1	Cifrari Storici	5
1.1	Shift Cipher	5
1.2	Cifrario per sostituzione	6
1.3	One Time Pad e Sicurezza Perfetta	6
1.4	Limitazioni della Perfetta Sicurezza	7
1.5	Teorema di Shannon	8
2	Cifratura a blocchi	9
2.1	Advanced Encryption Standard	9
3	Funzioni e Permutazioni Pseudo-Casuali	13
3.1	Esperimento Random vs Pseudo-Random	14
3.2	PRF implica sicurezza da Key Recovery	16
3.3	Attacco del compleanno	17
4	Cifrari Simmetrici	19
4.1	Modo ECB (Electronic Codebook Mode)	19
4.2	Modo CBC\$ (Cipher block Chaining Mode)	20
4.3	Modo CTR\$ (counter randomizzato)	20
4.4	Modo CTRC (modo counter non randomizzato)	21
4.5	Privacy	21
4.6	Indistinguibilità	22
4.7	Sicurezza di CTRC e CTRC\$	26
4.8	Attacchi a crittotesto scelto	28
5	Funzioni Hash	30
5.1	Applicazioni di funzioni Hash	32
5.2	SHA-1 (Secure Hash Algorithm 1)	33
5.3	Attacchi alle funzioni Hash	34
5.4	Sponge Construction	35
6	Message Authentication	37
6.1	Message Authentication Codes	37
6.2	MAC	38
6.3	Modello di attaccante	38
6.4	CBC-MAC	41
6.5	MAC da HASH (HMAC)	44

7	Primitive Asimmetriche	46
7.1	Un po' di Algebra	47
7.2	Primitive Asimmetriche	49
7.3	Calcolare Logaritmi Discreti	52
7.3.1	Baby-Step Giant-Step	52
7.3.2	Pohlig-Hellman	53
7.4	Problema della Fattorizzazione	54
7.4.1	RSA	54
7.4.2	Algoritmo Miller-Rabin	55
8	Cifrari Asimmetrici	58
8.1	Hybrid Encryption	60
8.2	Cifrario El-Gamal	61
8.2.1	Lifted El-Gamal	63
8.3	Cifrario Pailier	63
8.3.1	Un po' di Algebra (Ancora...)	64
8.3.2	Algoritmo di Cifratura	65
8.3.3	Elezione Multi-Candidato	67
8.4	Cifrari RSA-OAEP	68
8.5	Identity Based Encryption	70
8.5.1	Schema Boneh-Franklin	70
9	Firme Digitali	73
9.1	Firma RSA	74
9.2	Approccio Hash e Inverti	74
9.3	EC-DSA e Schorr's Signature	75

Introduzione

La crittografia è il processo di protezione delle informazioni o dei dati mediante l'uso di modelli matematici per codificarli di modo che solo le parti che hanno la chiave per decodificarli possano e accedervi. L'obiettivo di questo corso sarà quello di cercare di ottenere alcune proprietà di sicurezza su di un canale che di base è insicuro.

Punteremo quindi ad ottenere **privacy** tra i due interlocutori, per cui nessun altro potrà capire cosa si dicono, e **autenticità e integrità**, cioè che tutto ciò che proviene da un interlocutore viene certamente da lui e non sia stato alterato nel tragitto. Alla base di ciò, vi sta l'intuizione per cui vogliamo che l'avversario non solo non possa accedere alle informazioni trasmesse, ma che non possa modificarle né spacciarle per sue e, per fare ciò, è in genere prima necessario stabilire almeno un **segreto** in comune tra i due interlocutori. Il problema è proprio che è molto difficile stabilire in pubblico un segreto senza che i due interlocutori si siano mai visti prima.

Questo segreto è generalmente una chiave che può essere utile per **crittografia simmetrica** e **crittografia asimmetrica**. Una chiave simmetrica è un qualcosa che viene presa in input e trasforma un messaggio in un crittotesto. Colui che riceverà il crittotesto dovrà decifrarlo utilizzando la stessa chiave. Nel contesto asimmetrico abbiamo due chiavi, una che permette di fare la cifratura detta **chiave pubblica** e l'altra che permette di fare solo la de-cifratura detta **chiave privata**.

L'obiettivo della cifratura che effettueremo sarà quello di non fornire all'avversario informazioni aggiuntive rispetto a quelle precedenti alla trasmissione del messaggio. Immaginando di essere l'avversario che deve indovinare una risposta "Sì o No", avremo la probabilità del 50% di indovinarla prima ancora che la risposta sia trasmessa sul canale. Un buon sistema crittografico non deve aumentare questa probabilità, cioè l'attaccante non deve ricevere informazioni aggiuntive vedendo trasmettere il messaggio.

Quando analizzeremo i sistemi crittografici, dobbiamo avere bene in mente questi principi:

1. Per provare che qualcosa è sicuro, dobbiamo definire in modo chiaro e rigoroso cosa è **sicuro**;
2. Quando la sicurezza si basa su ipotesi non verificabile, questa deve essere espressa in modo chiaro;
3. Gli schemi crittografici dovrebbero essere corroborati da una dimostrazione di sicurezza matematicamente rigorosa.

Come sempre, supporremo che l'attaccante possa fare qualunque cosa ma che avrà una limitata potenza di calcolo che, di solito, imponiamo come uno dei computer più potenti al mondo. Dunque, potremo dire che, se l'attaccante ha una certa potenza di calcolo, allora il nostro sistema è sicuro.

Capitolo 1

Cifrari Storici

I cifrari storici sono tutti quei cifrari ideati prima degli anni 70. Questi sono quasi del tutto insicuri al giorno d'oggi. Prima di andare ad analizzare questi primi cifrari, diamo alcune definizioni utili:

- **Spazio delle chiavi K** : spazio contenente tutte le possibili chiavi del sistema crittografico;
- **Spazio dei messaggi M** : insieme di tutti i messaggi (o simboli) che si possono cifrare;
- **Spazio dei crittotesti C** : insieme di tutti i possibili crittotesti generabili dal sistema di cifratura;
- **Keygen**: funzione di generazione delle chiavi;
- **Encryption, $Enc(k,m)$ o Enc_k** : funzione che permette la cifratura del messaggio;
- **Decryption, $Dec(k,c)$** : funzione che permette la decifrazione di un crittotesto.

1.1 Shift Cipher

Il primo cifrario storico che analizzeremo è il cifrario per **Shift**. Per cui, avremo lo spazio delle chiavi K con 25 elementi e, per generare una chiave, ne decidiamo una a caso.¹ Dopodiché, associamo ad ogni lettera dell'alfabeto un numero (ad esempio $A = 0, B = 1, ecc...$) ed effettuiamo una *somma circolare*² tra il numero associato e la chiave. Fatto ciò, riportiamo i numeri nelle lettere corrispondenti e avremo ottenuto il crittotesto. Per decifrarlo procederemo semplicemente al contrario.

Notiamo a primo impatto come questo cifrario non sia sicuro poiché, anche andando per tentativi, si arriva facilmente a decifrare il testo poiché il numero di chiavi possibili è molto limitato. Capiamo subito quindi che un buon cifrario deve avere uno spazio delle chiavi grande abbastanza da non permettere di provare tutte le chiavi a casaccio.

¹Definiamo come **cifrario di Cesare**, il cifrario Shift con chiave uguale a 3.

²Somma per cui, una volta arrivati al massimo si riparte da 0.

1.2 Cifrario per sostituzione

L'idea alla base rimane quella di sostituire le lettere con altre. Cambiamo semplicemente lo spazio delle chiavi per cui ogni $k \in K$ sarà adesso una permutazione dei simboli $0, 1, 2, \dots, 25$ e per cifrare un simbolo, lo sostituiremo con il simbolo corrispondente nella chiave. Se $A \leftarrow K, C \leftarrow T, S \leftarrow D$, avremo che $CASA = TKDK$. Adesso la situazione cambia, il numero di chiavi possibili è molto alto e, considerando tutti i possibili caratteri ASCII, avremo un alfabeto di 256 caratteri, per cui avremo $256!$ chiavi che è circa 2^{896} e, anche considerando di poter elaborare 10^{12} chiavi al secondo, ci vorrebbero circa 14 miliardi di anni per trovare la chiave corretta.

Tuttavia, nonostante questo sembri rassicurarci, possiamo, tramite una semplice ricerca su internet, conoscere le statistiche che riguardano la presenza delle varie parole nelle parole della lingua italiana.

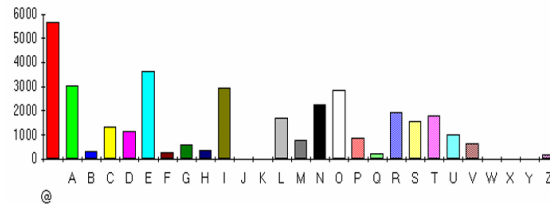


Figura 1.1: Frequenza delle varie lettere nel primo capitolo dei promessi sposi.

Grazie a queste informazioni, ci basta semplicemente andare a sostituire, in base alle frequenze trovate nel testo, le lettere con maggiore riscontro e, piano piano, con un testo sufficientemente grande, riusciremo a decifrare il testo.

1.3 One Time Pad e Sicurezza Perfetta

Sia $SE^3 = (\text{Keygen}, \text{Enc}, \text{Dec})$, diremo che è **Perfettamente sicuro** se

$$\forall m_1, m_2 \in M \quad \forall c \in C \quad \Pr[\text{Enc}(k, m_1) = c] = \Pr[\text{Enc}(k, m_2) = c]$$

4

Definiamo One Time Pad come:

- $K = M = C = \{0, 1\}^l$ questo schema ci indica che lo spazio delle stringhe è composto da $\{0, 1\}$ di lunghezza l ;
- $\text{KeyGen}(l) = K \xleftarrow{\text{random}} \{0, 1\}^l$ generiamo una chiave lunga l composta solo da 0 e 1;
- $\text{Enc}(K, M) = K \oplus M = C$;
- $\text{Dec}(K, M) = K \oplus C = M$

³Cifratura simmetrica.

⁴In questa definizione k è variabile.

Questo cifrario è *perfettamente sicuro* se la chiave viene utilizzata solo una volta. Possiamo dimostrare ciò verificando la condizione citata prima. Dati m_1, m_2 e c :

$$Pr[Enc_k(m_1) = c] = Pr_k[k \oplus m_1 = c] = Pr_k[k = m_1 \oplus c] = 1/2^l$$

$$Pr[Enc_k(m_2) = c] = Pr_k[k \oplus m_2 = c] = Pr_k[k = m_2 \oplus c] = 1/2^l$$

E $1/2^l$ corrisponde esattamente alla probabilità che k sia esattamente quella. Possiamo ancora testare questa proprietà per dimostrare che il cifrario per sostituzione non sia perfettamente sicuro. Per fare ciò, ci basta verificare che la proprietà non sia soddisfatta anche solo per una coppia da noi arbitrariamente scelta. Scegliamo $m_1 = \text{ciao}$ $m_2 = \text{dado}$ $c = \text{abcd}$ e su queste calcoliamo le probabilità.

$$Pr[Enc_k(m_1) = c] = Pr_k[Enc_k(\text{ciao}) = \text{abcd}] = \frac{22!}{26!}$$

$$Pr[Enc_k(m_2) = c] = Pr_k[Enc_k(\text{dado}) = \text{abcd}] = 0$$

Come possiamo vedere, la probabilità che m_2 sia cifrato in abcd è 0 poiché non è possibile raggiungere questa configurazione dato che ad una lettera se ne può associare solo un'altra e dado ha due lettere uguali. Possiamo anche dimostrare che One Time Pad è perfettamente sicuro solo se utilizziamo la chiave una sola volta. Nel caso di riutilizzo della chiave, un messaggio sarà una stringa $(x||y)^{\{0,1\}^{2l}} \oplus (k||k)^{\{0,1\}^l}$. Testando la proprietà:

$$m_1 = 0^{2l} \quad m_2 = 0^l || 1^l$$

$$Pr[Enc_k(m_1) = c] = 1/2^l \quad Pr[Enc_k(m_2) = c] = 0$$

Un ultimo test lo possiamo effettuare verificando cosa succederebbe se togliessimo la chiave composta da una serie di 0 su One Time Pad. A primo impatto ci sembra una scelta sicura poiché non rischiamo che il messaggio passi in chiaro. Verifichiamo la proprietà:

$$m_1 = m_2 \quad c = m_2$$

$$Pr[Enc_k(k, m_1) = c] = Pr_k[k \oplus m_1 = c] = Pr_k[k = m_1 \oplus c] = \frac{1}{2^l - 1}$$

$$Pr[Enc_k(k, m_2) = c] = Pr_k[k \oplus m_1 = c] = Pr_k[k = m_2 \oplus c] = 0$$

Dimostriamo che questo sistema, contro intuitivamente, non è perfettamente sicuro. Il punto è che, anche se il messaggio passasse realmente in chiaro, l'avversario non avrebbe modo di stabilirlo e, dunque, non gli sarebbe utile.

1.4 Limitazioni della Perfetta Sicurezza

Sia (KeyGen, Enc, Dec) uno schema perfettamente sicuro, M uno spazio dei messaggi e K lo spazio delle chiavi, allora $|K| \geq |M|$.

Supponiamo per assurdo che $|M| > |K|$ e che $c \in C$ sia un crittotesto raggiungibile almeno tramite una chiave.

$$M(c) = \{m : Dec_k(c) = m\}$$

Sappiamo che:

$$1 \leq |M(c)| \leq |K| \leq |M|$$

Per cui, prendiamo $m_2 \in M$, $m_2 \notin M(c)$ e $m_1 \in M(c)$. Avremo che:

$$Pr[Enc_k(m_1) = c] \neq 0$$

$$Pr[Enc_k(m_2) = c] = 0$$

1.5 Teorema di Shannon

SE = (KeyGen, Enc, Dec), $|M| = |K| = |C|$, SE offre perfetta sicurezza se e soltanto se:

1. $\forall k \in K$, K è scelta con probabilità $1/|K|$;
2. $\forall m \in M, \forall c \in C, \exists! k \in K$ tale che $Enc(k, m) = c$.

Capitolo 2

Cifratura a blocchi

In crittografia un algoritmo di cifratura a blocchi è un algoritmo a chiave simmetrica operante su un gruppo di bit di lunghezza finita organizzati in un blocco. A differenza degli algoritmi a flusso che cifrano un singolo elemento alla volta, gli algoritmi a blocco cifrano un blocco di elementi contemporaneamente.

$$E : \{0, 1\}^k * \{0, 1\}^n \rightarrow \{0, 1\}^m$$

E questa deve godere delle seguenti proprietà:

1. Invertibile una volta fissata una chiave;
2. Deve essere efficace e veloce;
3. E deve essere "sicuro".

DES fu il primo cifrario a blocchi nato per scopi militari e questo, dovendo funzionare solo su specifici hardware, risultava parecchio performante per gli hardware su cui si basava ma le performance calavano quando dall'hardware si passava al software. Per DES la chiave k era composta da 56 bit mentre il blocco era composto da 64 bit.¹ A causa del fatto che questo algoritmo avesse basse performance dal punto di vista software, si decise di sostituirlo andando alla ricerca di un cifrario con dimensioni di chiave e blocchi maggiori e più veloci. Per fare ciò venne indetto un concorso pubblico e, tra tutti gli algoritmi di cifratura proposti, la scelta ricadde sull'algoritmo Rijndael che poi divenne lo standard **AES**.

2.1 Advanced Encryption Standard

Questo algoritmo di cifratura a chiave simmetrica fu quello che sostituì totalmente l'algoritmo DES e che fu' adottato pure dai servizi segreti americani per cifrare documenti top secret. AES si divide in tre livelli fondamentali detti **layers** e, ognuno di questi, manipola tutti i bit dello *stato*.² Avremo:

¹Vedremo poi perché queste scelte sono problematiche.

²Lo stato è un valore inizialmente inizializzato a M e che come stato finale il crittotesto C .

- **Key addition layer:** in cui lo stato attuale viene aggiornato usando una chiave di round di 128 bit. Questa chiave in realtà è composta da 11 sotto-chiavi che vedremo dopo nello specifico come vanno ad agire;
- **Byte Substitution layer:** lo stato è modificato da operazioni non lineari. È importante che non lo siano poiché altrimenti basterebbe risolvere un sistema lineare per rompere il cifrario;
- **Diffusion layer:** diffonde tramite trasformazioni lineari tutti i bit dello stato. Si basa su due procedure che non sono altro che scambio di righe e scambio di colonne;

```

AESk(M) // |M|=128 e |K|=128
(K0,...,K10) ← Espandi_Chiave(K) // |Ki|=128
s ← M ⊕ K0 // s e' chiamato lo stato
for r=1 to 10 do
    s ← S(s)
    s ← Shift_Rows(s)
    if r ≤ 9 then mix_cols(s)
    s ← s ⊕ Kr
Return s

```

Figura 2.1: Pseudo-codice dell'algoritmo AES.

La prima funzione che salta all'occhio è quella di **espansione** che non fa altro che prendere in input la chiave K di 128 bit per poi produrre 11 sotto chiavi di 128 bit ciascuna. Prima di andare a vedere la procedura S , è bene dare alcune definizioni utili.

Sappiamo che possiamo rappresentare 1 byte come un generico polinomio di questo tipo $a_7x^7 + a_6x^6 + \dots + a_1x + a_0$. Per poter lavorare concretamente con questo polinomio, abbiamo bisogno di due operazioni che rispettino alcune proprietà:

1. Vogliamo che la somma sia chiusa, per cui se facciamo una somma, il risultato non esce dall'insieme di riferimento;
2. Vogliamo che valgano le proprietà **commutative** e **associativa**;
3. Vogliamo l'esistenza di un elemento neutro;³
4. Ogni elemento deve avere un inverso;⁴

Quando esistono due operazioni di questo tipo, allora parleremo di un **campo**. Nel nostro caso, avremo come operazioni:

- $\oplus$ **XOR**: somma classica tra polinomi ma il tutto viene fatto modulo 2. Non a caso, per implementarlo viene utilizzato uno xor vero e proprio che è un'operazione estremamente efficiente in termini di calcolo. Esempio:

$$x^7 + x^6 + x^4 + x + 1 \oplus x^6 + x^5 + x^4 + x = x^7 + x^5 + 1$$

³Se viene eseguita un'operazione tra un elemento e il neutro, l'elemento rimane invariato.

⁴Se viene eseguita un'operazione tra un elemento e il suo inverso, il risultato sarà l'elemento neutro.

- *** moltiplicazione:** facciamo la tipica moltiplicazione tra polinomi e, nel caso in cui si andasse in un polinomio oltre il grado 7 che quindi non rientrerebbe in un byte, effettuiamo una riduzione con un polinomio di grado 8 che, per AES, è $x^8 + x^4 + x^3 + x + 1$. Esempio:

$$(x^7 + x^6 + x^4 + x + 1) * (x^6 + x^5 + x^4 + x = x^7 + x^5 + 1) = x^{13} + x^9 + x^5 + x^4 + x^2 + x$$

Come già detto questo non rientra in un byte, dunque, facendo una semplice divisione tra polinomi, otteniamo $x^6 + x^3 + x^2 + 1$. In AES le moltiplicazioni si fanno tendenzialmente per potenze di due per cui l'operazione di moltiplicazione non è altro che uno shift a sinistra che, nel caso in cui si ecceda con il grado del polinomio, basta effettuare l'operazione di riduzione. Tuttavia, potendo eccedere solo al grado 8, la riduzione consisterà semplicemente in uno xor con il polinomio riduttore di AES.

In base a quanto detto, vedremo che l'operazione S di AES non è altro che una sequenza, seppur lunga, di shift e xor che sono operazioni efficientissime da implementare.

L'operazione S è composta da due operazioni principali:

- **inv'(a):** un'operazione in grado di trovare l'inverso di a. Vedremo poi che esistono dei modi particolari per ottimizzare questo processo;
- **aff(a):** un'operazione non lineare che consiste sostanzialmente in una moltiplicazione riga per colonna e poi una somma;

$$x'_0 x'_1 x'_2 x'_3 x'_4 x'_5 x'_6 x'_7 = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

Questo processo potrebbe ottimizzato ancora di più facendo alcune semplici considerazioni. Nel campo di Galois sono presenti alcuni elementi detti **generatori** che permettono, moltiplicandoli per sé stessi, di generare tutti i numeri del campo. Continuando con le potenze varrà sempre la proprietà $g^{255} = 1$ cioè l'elemento neutro della moltiplicazione da cui poi si ripartirà da capo se si continua.

Possiamo tabellare tutti i valori⁵ calcolati dalla potenza di un generatore memorizzando potenza-risultato e, sfruttando le proprietà delle potenze se $a = g^x$, allora $a^{-1} = g^{255-x}$. Così facendo, trasformiamo l'operazione d'inverso in una operazione di look-up sulla tabella.

⁵La tabellazione di questi valori verrà fatta in momenti in cui il processore è libero, così da avere costo 0.

Un'ulteriore ottimizzazione la si può ottenere tabellando in maniera diretta i risultati della funzione S box dato che questi sono costanti. Tuttavia, questa tabella può essere utilizzata solo se si ha la certezza che non potrà essere modificata quando in memoria, quindi bisogna stare attenti a quando implementare questa funzione.

Per quanto riguarda invece la funzione shift-rows e shift-cols, si tratta di disporre gli stati in una matrice⁶ per poi effettuare degli scambi. Per quanto riguarda shift-rows la prima riga rimane invariata, la seconda fa uno shift a sinistra di una posizione, la terza shift di due posizioni a sinistra e così via. La mix-cols fa più o meno la stessa cosa.

Quello che vogliamo adesso è trovare una proprietà di sicurezza che ci garantisce in maniera certa che il sistema è sicuro. Verifichiamo cosa succede se il sistema è sicuro rispetto a un **attacco di Key Recovery**. Immaginiamo di essere un attaccante e di poter vedere blocco di input e blocco di output e di poter mandare noi stessi i messaggi. Infine, analizzando questi, vogliamo trovare la chiave. AES è sicuro rispetto a questo tipo di attacco ma domandiamoci, è una condizione sufficiente a dire che il nostro sistema è sicuro? Prendiamo un cifrario $E_k(x) = x$, questo è sicuro da attacchi di key recovery dato che, banalmente, la chiave non la stiamo utilizzando. Capiamo quindi che questa proprietà è sì necessaria, ma non è sufficiente a dire che il nostro sistema è sicuro. Proviamo adesso a verificare se il sistema è sicuro rispetto ad **attacchi di Plain-Text Recovery**. In questo scenario l'attaccante avrà a disposizione esclusivamente i messaggi cifrati e cercherà di decifrarli non necessariamente estraendo la chiave. Anche in questo caso, AES è sicuro da questo tipo di attacchi, tuttavia anche questa è una condizione necessaria ma non sufficiente poiché un ipotetico sistema che cifra tutti i bit tranne gli ultimi 10, rispetta la condizione, ma non è sicuro.

⁶Dall'alto verso il basso

Capitolo 3

Funzioni e Permutazioni Pseudo-Casuali

In crittografia una componente fondamentale sono le funzioni **PRF** e le permutazioni **PRP** pseudo-casuali. Definiamo, innanzitutto, una famiglia di funzioni del tipo:

$$F : K \times D \rightarrow R$$

Dove per noi K sarà l'insieme delle chiavi, D il dominio e R il codominio e, ognuno di questi, è un insieme finito e non vuoto. Diremo inoltre che F_k è un'istanza di F con una specifica chiave. Per capire meglio questo concetto, immaginiamo che l'input sia "CASA" e l'output sia "DERE", in questo caso la funzione F_k sarà la funzione per cui k assegna $(C \rightarrow D, A \rightarrow E, S \rightarrow R)$. Diamo alcune definizioni:

- $K = \{0, 1\}^k$, k intero che stabilisce la lunghezza della chiave.
- $D = \{0, 1\}^l$, l intero che stabilisce la lunghezza dell'input.
- $R = \{0, 1\}^L$, L intero che stabilisce la lunghezza dell'output.
- $\xleftarrow{R}$, indica la scelta casuale di qualcosa.

Per quanto riguarda una permutazione π è una biezione nella quale $D=R$ per cui, per ogni x in D esiste solo un y in R tale che $\pi(x) = y$. Consideriamo:

$$Func(D, R) \quad D = \{0, 1\}^l \quad R = \{0, 1\}^L$$

Avremo perciò una mappatura per cui $(x_1, x_2, \dots, x_t) \rightarrow (y_1, y_2, \dots, y_t)$ dove $t = 2^l$ per cui, se ad ogni input corrisponde una stringa di L bit, avremo bisogno in totale per memorizzare una funzione $2^l * L$ bit e, se volessimo considerare tutte le possibili funzioni sarebbero $2^{2^l * L}$ che è un numero impossibile da raggiungere. Per cui già da qui possiamo immaginare un limite, $2^{128} * 128$ bit sono all'incirca 5.44×10^{24} Petabyte di spazio necessari solo a memorizzare una singola funzione casuale e, ciò, non è chiaramente fattibile.

Supponiamo di avere una di queste funzioni e, supponiamo che a partire da un input e un output, vogliamo prevedere come potrebbe comportarsi questa funzione.

$$Pr[f(x) = y] = \frac{2^{(2^l - 1)L}}{2^{2^l L}} = \frac{1}{2^L}$$

¹ Se continuassimo ad analizzare questo processo, scoprendo man mano sempre più rapporti di tipo input-output, la probabilità di indovinare il comportamento di questa funzione, rimarrebbe comunque di $\frac{1}{2^l}$ che, nel pratico, vuol dire che continuiamo a sparare a caso. Ciò è dovuto al fatto che i valori di una funzione casuale non dipendono in alcun modo da quelli precedenti e, dunque, non ricaviamo alcuna informazione utile.

Analizziamo invece il caso in cui questo esperimento si ripeti con una permutazione. In quel caso avremo una qualche tipo di relazione perché, sapendo che se un comportamento y_i si è avuto in un certo x_i , allora y_i non potrà più essere generato. Andiamo ad analizzare le probabilità.

$$Pr[\pi(x_2) = y_2 | \pi(x_1) = y_1] = \frac{(2^l - 2)!}{(2^l - 1)!} = \frac{1}{2^l - 1}$$

Per quanto quel -1 possa far sembrare le probabilità a nostro favore, scopriremo con tanta delusione che, per quanto 2^l sia grande, non sarà praticamente cambiato nulla.

Le funzioni casuali, come abbiamo dimostrato, non sono in alcun modo memorizzabili in alcun hard disk o ssd esistente. Per ciò abbiamo bisogno di funzioni **Pseudo-casuali** che possiedono una rappresentazione compatta e che, sotto certe condizioni, sembrino in tutto e per tutto indistinguibili da una funzione casuale. Le condizioni in questione sono:

1. Potenza di calcolo limitata.
2. Approccio black-box, per cui non si può vedere come la funzione agisca.

3.1 Esperimento Random vs Pseudo-Random

Sia $F : K \times D \rightarrow R$ una famiglia di funzioni e A un avversario che ha accesso black box alla funzione in questione. In questo esperimento l'attaccante dovrà capire se il risultato che riceve deriva da una funzione random o da una funzione pseudo-random.

$\text{Esp}_F^{\text{prf-1}}(A)$ $K \leftarrow_R K$ $b \leftarrow A^K$ Return b // Funz. PseudoCasuale	$\text{Esp}_F^{\text{prf-0}}(A)$ $g \leftarrow_R \text{Func}(D, R)$ $b \leftarrow A^g$ Return b // Funz. Casuale
--	--

Definiremo vantaggio $Adv(A) = |Pr[\text{Esp}_F^{\text{prf-1}}(A) = 1] - Pr[\text{Esp}_F^{\text{prf-0}}(A) = 1]|$, più questo valore è piccolo, più la prf è sicura. Proviamo questa funzione

¹ 2^l sono tutti i possibili output per cui al numeratore troviamo un -1 perché ne stiamo fissando uno

$$F : K \times \mathcal{D} \rightarrow \mathbb{R} \quad K = \{0, 1\}^{\ell \cdot L} \quad \mathcal{D} = \{0, 1\}^{\ell} \quad \mathbb{R} = \{0, 1\}^L$$

$$F_K(x) = \begin{bmatrix} k(1,1) & \dots & k(1,\ell) \\ \vdots & & \vdots \\ k(L,1) & \dots & k(L,\ell) \end{bmatrix} \odot \begin{bmatrix} x_1 \\ \vdots \\ x_\ell \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_L \end{bmatrix}$$

$$y_i = (k(i,1) \wedge x_1) \vee (k(i,2) \wedge x_2) \vee \dots \vee (k(i,\ell) \wedge x_\ell)$$

E testiamo con la definizione che abbiamo dato. Supponiamo di avere l'attaccante:

Algorithm 1 Algoritmo $A(F)$

```

1:  $x \leftarrow 0^l$  ▷ Stringa di  $l$  zeri
2:  $y \leftarrow F(x)$  ▷ Interroga l'oracolo  $F$ 
3: if  $y == 0^l$  then
4:   return 1
5: else
6:   return 0
7: end if

```

Allora possiamo verificare le probabilità:

$$Pr[Exp^{prf-1}(A) = 1] = 1 \quad Pr[Exp^{prf-0}(A) = 1] = 1/2^L$$

Il vantaggio è $Adv(A) = 1 - 1/2^L$

Per continuare ad esercitarci, immaginiamo adesso di avere utilizzare una prf che è praticamente one-time-pad che però viene usata più volte e creiamo un attaccante.

Algorithm 2 Algoritmo $A(F)$

```

1:  $x_1 \leftarrow 0^l$  ▷ Stringa di  $l$  zeri
2:  $y_1 \leftarrow F(x_1)$  ▷ Ritorna la chiave  $k$ 
3:  $x_2 \leftarrow y_1$ 
4:  $y_2 \leftarrow F(x_2)$ 
5: if  $y_2 == 0^l$  then ▷ La chiave xor sé stessa fa  $0^l$ 
6:   return 1
7: else
8:   return 0
9: end if

```

Per cui, le probabilità sono $Pr[Exp^{prf-1}(A) = 1] = 1$ $Pr[Exp^{prf-0}(A) = 1] = 1/2^l$ e il vantaggio è $Adv(A) = 1 - 1/2^l$.

3.2 PRF implica sicurezza da Key Recovery

Data questa definizione, abbiamo capito che effettivamente la proprietà di pseudo-randomicità è sintomo di sicurezza. Tuttavia il problema principale di ciò è che non possiamo dimostrare che un cifrario sia pseudo-casuale. Quello che faremo sarà assumere che il cifrario lo sia e, sulla base di ciò agiremo. L'idea è che legheremo la pseudo casualità ad un altro grande problema dell'informatica e faremo sì che se si riesce a rompere quello, allora si risolve il problema².

Supponiamo di voler dimostrare che la PRF implichi sicurezza da key recovery in maniera diretta. Prima di tutto facciamo sì che tutto sia ricondotto agli stessi termini così da poterne fare il confronto. Definiamo B, l'attaccante che si occuperà di attaccare il cifrario per Key recovery.

$F: K \times D \rightarrow R$, famiglia di funzioni.

```

EspFkr (B)
k  $k \leftarrow_R K$ 
 $K' \leftarrow B^{F_k}$ 
If  $K'=K$  Return 1 else return 0
  
```

$$\text{Adv}(B) = \Pr[\text{Esp}_F^{\text{kr}}(B) = 1]$$

E, indicando con A l'attaccante che attacca il cifrario per indistinguibilità, vogliamo che

$$\text{Adv}(B) \leq \text{Adv}(A)$$

Questa è molto interessante poiché notiamo come se l'attacco di key recovery funziona, allora l'attacco per indistinguibilità deve essere almeno buono tanto quello ma, se l'attacco per indistinguibilità è uguale a 0, allora lo sarà anche l'altro.

Descriviamo l'attaccante A:

```

i=0;
Run B
  when B asks for x do
    { i++; xi = x;
      yi = OA(xi);
      Return yi to B; }
  until B stops and outputs k'
  Let x ←R D - {x1, ..., xq}
  y = OA(x);
  If (F(k', x) == y) return 1
  else return 0
  
```

Patricio

DEVE ESSERE DIVERSO
DA QUELLO CHE HA
GIÀ SCUOTO B

Dimostriamo che quella relazione vale sempre.

$$\Pr[\text{Esp}^{\text{prf}-1}(A) = 1] \geq \text{Adv}(B)$$

²Problema che non è mai stato risolto e, probabilmente, non lo sarà mai.

Può accedere se B è bravo e quindi io dico 1, o se B spara a caso e sbaglio e io, per puro caso, ho l'if vero. Invece:

$$Pr[Esp^{prf-0}(A) = 1] = \frac{1}{|R|}$$

Da questo, segue che

$$Pr[Esp^{prf-1}(A) = 1] - Pr[Esp^{prf-0}(A) = 1] \geq Adv(B) - 1/|R|$$

3.3 Attacco del compleanno

Supponiamo di avere $E : \{0, 1\}^k \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ una famiglia di permutazioni. Supponiamo di ricevere un accesso black box ad una funzione casuale g che è o un'istanza di E o una funzione casuale. Immaginiamo di voler distinguere i due casi, possiamo invocare la funzione su q punti e, se g è una permutazione, ci basterà verificare che questa sia diversa per ogni punto. A primo impatto questo attacco sembra essere poco efficace, d'altro trovare tutte le possibili combinazioni non è fattibile. Tuttavia, vedremo che in realtà le probabilità sono molte più di quelle che possiamo immaginare grazie a quello che viene definito **paradosso del compleanno**. Di fatto, vedremo che basteranno più o meno $q = \sqrt{2^l}$ domande per avere un vantaggio prossimo a 1.

Il motivo per cui accade ciò è banalmente da imputare ad un dettaglio fondamentale della domanda. Noi non vogliamo trovare una specifica collisione, a noi ce ne basta una tra tutte le possibili coppie e non tra una specifica coppia.

Supponiamo di avere q palline e N buche e, in base a ciò, supponiamo di voler valutare la probabilità che due palline finiscano nella stessa buca considerando che ogni pallina può finire in una qualsiasi delle buche con la stessa probabilità. Naturalmente se le buche sono meno delle palline per il teorema del Pigeon Hole questo sarà un evento certo, per cui, valutiamo il caso in cui $N \gg q$. Valutiamo gli eventi più semplici, sia D_i l'evento che non vi sia alcuna collisione dopo aver lanciato

l'iesima pallina.

$$Pr[D_1] = 1 \quad Pr[D_{i+1}|D_i] = \frac{N-1}{N}$$

Proviamo a valutare D_q

$$\begin{aligned} Pr[D_q] &= Pr[D_q \wedge D_{q-1}] + Pr[D_q \wedge D_{q-1}] = \\ &= Pr[D_q|D_{q-1}]Pr[D_{q-1}] + Pr[D_q|D_q]Pr[!D_{q-1}] = \\ &= Pr[D_q|D_{q-1}]Pr[D_{q-1}] = \\ &= Pr[D_q|D_{q-1}]Pr[D_{q-1}|D_{q-2}] \dots Pr[D_2|D_1]Pr[D_1] = \\ &= \prod_{i=1}^q \left(\frac{N-i}{N}\right) = \prod_{i=1}^q \left(1 - \frac{i}{N}\right) \sim e^x \quad x_i = i/N \quad 0 < \frac{i}{N} < 1 \\ e^x &= 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots \\ e^{-x} &= 1 - x + \frac{x^2}{2!} - \frac{x^3}{3!} \\ e^{-x} &\sim 1 - x \\ &\sim \prod_{i=1}^q e^{-\frac{1}{N}} = e^{-\sum_{i=1}^q \frac{i}{N}} = e^{-\frac{1}{N} \sum_{i=1}^q i} \sim e^{-\frac{q^2}{2N}} \end{aligned}$$

Il nostro obiettivo è far sì che $Pr[!D_q] \geq 1/2$ per cui possiamo continuare così:

$$\begin{aligned} \frac{1}{2} &= 1 - e^{-\frac{q^2}{2N}} \\ &= \frac{1}{2} = e^{-\frac{q^2}{2N}} \\ &= \ln\left(\frac{1}{2}\right) = -\frac{q^2}{2N} \rightarrow q = \sqrt{2\ln(2N)} \end{aligned}$$

Dunque, un cifrario a blocchi³ può essere distinto da una funzione casuale guardando un numero $2^{l/2}$ di coppie input-output.

³I cifrari a blocchi sono considerati permutazioni perché, per una data chiave segreta, stabiliscono una corrispondenza biunivoca tra tutti i possibili blocchi di testo in chiaro e tutti i possibili blocchi di testo cifrato.

Capitolo 4

Cifrari Simmetrici

Un cifrario simmetrico è composto principalmente da tre cose:

- **KeyGen:** Algoritmo randomizzato di generazione di una chiave da un insieme K .
- **Enc:** Algoritmo di Encryption a stati o randomizzato che prende in input una chiave k e un messaggio m e restituisce un crittotesto C .
- **Dec:** Algoritmo deterministico in grado di ritornare al messaggio m a partire da (k, C) .

Un cifrario simmetrico è **randomizzato** se utilizza una qualche quantità di randomness. Diciamo, invece, che un cifrario è a stati se il risultato dipende da una certa quantità detta **stato**. Questi cifrari utilizzano meccanismi di cifratura a blocchi e, per semplicità, supporremo che la dimensione del messaggio è un multiplo della taglia del blocco.

4.1 Modo ECB (Electronic Codebook Mode)

Sia $E : K \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ un cifrario a blocchi, ECB produce un cifrario senza stati e deterministico. L'algoritmo di generazione della chiave si limita a restituire una stringa random e, per effettuare Encryption e Decryption utilizziamo i seguenti algoritmi:

```
Enck(M)
  if ( $|M| \bmod n \neq 0$ )  $\vee$  ( $|M| = 0$ ) return  $\perp$ 
  Sia  $M = M[1] \dots M[m]$  ( $|M[i]| = n$ )
  for  $i = 1$  to  $m$  do
     $C[i] = E_k(M[i])$ 
   $C \leftarrow C[1] \dots C[m]$ 
  return  $C$ 


---


Deck(C)
  if ( $|C| \bmod n \neq 0$ )  $\vee$  ( $|C| = 0$ ) return  $\perp$ 
  Sia  $C = C[1] \dots C[m]$ 
  for  $i = 1$  to  $m$  do
     $M[i] = E_k^{-1}(C[i])$ 
   $M \leftarrow M[1] \dots M[m]$ 
  return  $M$ 
```


Questo algoritmo non fa altro che dividere il messaggio in vari blocchi ed effettuare encryption blocco per blocco in base a una chiave scelta a random. Allo stesso modo, a partire dalla chiave, si effettua la decryption del messaggio.

4.2 Modo CBC\$ (Cipher block Chaining Mode)

Questo modo è **randomizzato** e lo possiamo denotare dalla presenza del \$ nel nome che, d'ora in avanti, indicherà tutti i cifrari randomizzati. Sia $E : K \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ un cifrario a blocchi, CBC\$ genera un vettore iniziale random detto **IV** e, tramite questo, effettua la cifratura del messaggio. In questo caso, la chiave sarà semplicemente una stringa random di una certa lunghezza. Viene, inoltre, definito **Chain model** perché una singola decisione, cambia tutta la catena di cifratura.¹

```

Enck(M)
  if ( $|M| \bmod n \neq 0$ )  $\vee$  ( $|M| = 0$ ) return  $\perp$ 
  Sia  $M = M[1] \dots M[m]$  ( $|M[i]| = n$ )
   $C[0] \leftarrow \text{IV} \leftarrow_R \{0, 1\}^n$ 
  for  $i = 1$  to  $m$  do  $C[i] = E_k(C[i-1] \oplus M[i])$ 
   $C \leftarrow C[1] \dots C[m]$ 
  return  $\langle \text{IV}, C \rangle$ 


---


Deck( $C, \text{IV}$ )
  if ( $|C| \bmod n \neq 0$ )  $\vee$  ( $|C| = 0$ ) return  $\perp$ 
  Sia  $C = C[1] \dots C[m]$ 
   $C[0] \leftarrow \text{IV}$ 
  for  $i = 1$  to  $m$  do  $M[i] = E_k^{-1}(C[i]) \oplus C[i-1]$ 
   $M \leftarrow M[1] \dots M[m]$ 
  return  $M$ 

```

Anche in questo caso l'algoritmo divide il tutto in blocchi, dopodiché, imposta una cifratura a catena con $C[0] = \text{IV}$ e, i successivi blocchi $C[i]$ saranno $C[i] = E_k(C[i-1] \oplus M[i])$. Come possiamo notare, è necessario ritornare anche il vettore IV per poter effettuare la decifrazione. Per decifrare il tutto faremo l'operazione inversa sempre a partire dello stesso $C[0]$ poiché $M[i] = C[i-1] \oplus E_k^{-1}(C[i])$.

4.3 Modo CTR\$ (counter randomizzato)

Sia $F : K \times \{0, 1\}^l \rightarrow \{0, 1\}^L$ una famiglia di funzioni, CTR\$ produce un cifrario randomizzato senza stati. Anche in questo caso, la chiave sarà semplicemente una stringa random. E, una precisazione necessaria da fare è che in questo specifico modo non abbiamo la necessità che la dimensione del messaggio sia multiplo del blocco.

¹In questo caso la decisione è il vettore iniziale IV

```

Enck(M)
   $m \leftarrow \lceil |M|/L \rceil$ ;  $R \leftarrow_R \{0, 1\}^\ell$ 
   $Pad \leftarrow F_k(R+1) || F_k(R+2) || \dots || F_k(R+m)$ 
   $Pad \leftarrow \text{Primi } |M| \text{ bits di } Pad$ 
   $C' \leftarrow M \oplus Pad$ 
   $C \leftarrow R || C'$ 
  return  $C$ 


---


Deck(C)
  if  $(|C| < \ell)$  return  $\perp$ 
  Sia  $C = R || C'$  ( $|R| = \ell$ )
   $m \leftarrow \lceil |C'|/L \rceil$ 
   $Pad \leftarrow F_k(R+1) || F_k(R+2) || \dots || F_k(R+m)$ 
   $Pad \leftarrow \text{Primi } |C'| \text{ bits di } Pad$ 
   $M \leftarrow C' \oplus Pad$ 
  return  $M$ 

```

Quello che facciamo è dividere anche in questo caso il messaggio in "blocchi" e, successivamente, generiamo $Pad = F_k(R+1) || F_k(R+2) || \dots || F_k(R+m)$ dove R è una stringa random che funge da componente random². Una volta generato il padding, prendiamo solo i bit che ci servono e facciamo uno xor tra il messaggio M e Pad . Anche in questo caso, come prima, abbiamo la necessità di salvare in qualche modo la chiave R che, semplicemente, concateniamo al crittotesto. Per quanto riguarda la decryption, generiamo nuovamente il padding e rifacciamo lo xor con il crittotesto, ottenendo nuovamente il messaggio.

4.4 Modo CTRC (modo counter non randomizzato)

Questo modo non è altro che CTR\$ che, invece di essere randomizzato è a **stati**. In pratica, teniamo un contatore globale ctr , inizialmente a 0, che utilizziamo allo stesso modo di come utilizzavamo prima la componente random.

```

Enck(M)
   $m \leftarrow \lceil |M|/L \rceil$ ;
  If  $ctr+m \geq 2^\ell$  return  $\perp$ 
   $Pad \leftarrow F_k(ctr+1) || F_k(ctr+2) || \dots || F_k(ctr+m)$ 
   $Pad \leftarrow \text{Primi } |M| \text{ bits di } Pad$ 
   $C' \leftarrow M \oplus Pad$ 
   $ctr \leftarrow ctr+m$ 
  return  $\langle ctr-m, C' \rangle$ 


---


Deck( $\langle i, C \rangle$ )
   $m \leftarrow \lceil |C|/L \rceil$ ;
   $Pad \leftarrow F_k(i+1) || F_k(i+2) || \dots || F_k(i+m)$ 
   $Pad \leftarrow \text{Primi } |C| \text{ bits di } Pad$ 
   $M \leftarrow C \oplus Pad$ 
  return  $M$ 

```

4.5 Privacy

Come facciamo, adesso visti alcuni esempi, a scegliere quale cifrario utilizzare e quale è il migliore rispetto a un altro cifrario. Il nostro obiettivo sarà quelli di definire

²NB: Ricordiamo che R è diverso dalla chiave

in maniera soddisfacente la sicurezza o cosa rappresenta la privacy di un sistema di questo tipo così da poter stabilire un metodo di confronto tra i vari modi.

Chiaramente a primo impatto una definizione che sembra funzionare è che se A può dedurre M da C , allora lo schema non è sicuro. Tuttavia, questa cosa non funziona ugualmente poiché anche alcune informazioni parziali potrebbero fare la differenza. Abbiamo, dunque, bisogno di una definizione più forte esattamente come abbiamo fatto per i cifrari a blocchi.

Idealmente vorremmo che il nostro testo cifrato non riveli alcuna informazione riguardo al messaggio M , tuttavia alcune informazioni a priori sono inevitabili da mostrare come, ad esempio, la dimensione del messaggio poiché non possiamo comprimere il messaggio più di una certa dimensione e non possiamo allungare il messaggio più di tanto perché rischieremmo di floodare la rete. Tuttavia, analizzando ECB può giungerci un suggerimento. Come abbiamo visto ECB è un cifrario del tutto deterministico, per cui, cifrando due blocchi esattamente uguali, vedremo esattamente lo stesso risultato. Ergo, la proprietà che sceglieremo sarà proprio quella di **indistinguibilità** rispetto ad attacchi a **testo scelto**.

4.6 Indistinguibilità

Immaginiamo il seguente esperimento: l'attaccante sceglie una coppia M_0, M_1 che manderà ad un oracolo che sceglierà se cifrare solo il M_0 o solo M_1 . L'obiettivo dell'attaccante sarà quello di riuscire a capire quale dei due messaggi è stato cifrato a patto che $|M_0| = |M_1|$. L'attaccante potrà fare più domande anche basate sulle risposte precedenti sempre a patto che le domande siano computazionalmente fattibili.

$\mathcal{SE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ cifrario simmetrico

$\text{Esp}_{\mathcal{SE}}^{\text{ind-cpa-1}}(A)$	$\text{Esp}_{\mathcal{SE}}^{\text{ind-cpa-0}}(A)$
$K \leftarrow_R \text{KeyGen}$	$K \leftarrow_R \text{KeyGen}$
$b \leftarrow A^{\text{Enc}_K(\text{LR}(\cdot, \cdot, 1))}$	$b \leftarrow A^{\text{Enc}_K(\text{LR}(\cdot, \cdot, 0))}$
Return b	Return b

Anche in questo caso possiamo definire il vantaggio dell'attaccante A come:

$$\text{Adv}^{\text{ind-cpa}}(A) = |Pr[\text{Esp}_{\mathcal{SE}}^{\text{ind-cpa-1}}(A) = 1] - Pr[\text{Esp}_{\mathcal{SE}}^{\text{ind-cpa-0}}(A) = 1]|$$

Se l'avversario può vincere con probabilità $1/2$ questo non è considerabile una minaccia poiché equivarrebbe al tentare a casaccio. Il vantaggio più sarà prossimo a 1 più l'attaccante avrà probabilità di dare la risposta corretta.

Equivalentemente all'esperimento precedente, possiamo scrivere:

SE=(KeyGen, Enc, Dec) cifrario simmetrico

$\text{Esp}_{SE}^{\text{ind-cpa-cg}}(A)$
 $b \leftarrow_R \{0,1\}; K \leftarrow_R \text{KeyGen};$
 $b' \leftarrow_R A^{\text{Enc}_K(\text{LR}(\cdot, b))};$
 If $b'=b$ return 1 else return 0

E in questo caso il vantaggio cambierebbe in:³

$$\text{Adv}^{\text{ind-cpa}}(A) = 2\Pr[\text{Esp}_{SE}^{\text{ind-cpa-cg}}(A) = 1] - 1$$

Tornando a ECB, ci possiamo facilmente rendere conto come fallisca in questo specifico esperimento poiché possiamo costruire un'attaccante in grado di rispondere con certezza quale dei due messaggi è stato cifrato:

$A(SE^{\text{ECB}})$
 $X, Y \leftarrow \{0,1\}^m \parallel X \neq Y$
 $M_0 \leftarrow X \parallel X; M_1 \leftarrow X \parallel Y$
 $C \leftarrow \text{Enc}(M_0, M_1)$
 $C = C[1] \parallel C[2]$
 If $(C[1] \neq C[2])$ return 1
 else return 0

$\Pr[\text{Esp}^{\text{ind-cpa-1}}(A) = 1] = 1 \quad \text{Adv}(A) = 1$
 $\Pr[\text{Esp}^{\text{ind-cpa-0}}(A) = 1] = 0$

Da qui possiamo notare come, in realtà, qualunque schema di cifratura deterministico e senza stati sia del tutto insicuro. Proviamo adesso a dimostrare che se un eventuale attacco di plain text recovery funziona, allora rompe lo schema anche in senso di indistinguibilità. Partiamo semplicemente da un cifrario simmetrico senza stati e facciamo il seguente esperimento:

$\text{Esp}_F^{\text{pr-cpa}}(B)$
 $K \leftarrow_R \text{KeyGen}; M' \leftarrow_R \{0,1\}^m$
 $C \leftarrow_R \text{Enc}_K(M')$
 $M \leftarrow B^{\text{Enc}_K(\cdot)}(C)$
 If $M=M'$ Return 1 else return 0

$$\text{Adv}(B) = \Pr[\text{Esp}_{SE}^{\text{pr-cpa}}(B) = 1]$$

³Si può dimostrare e la dimostrazione è presente nelle slide

Per cui, l'obiettivo di B sarà quello di indovinare il testo del messaggio cifrato a partire dalla sua cifratura potendo fare domande del tipo a questo input quale output corrisponde. Il suo vantaggio sarà in questo caso:

$$Adv(B) = Pr[Esp_{SE}^{pr-cpa}(B) = 1]$$

Ragionando su questo vantaggio, dobbiamo tenere conto del fatto che esiste un numero limitato di messaggi che si possono generare. Per cui, diremo che un buon vantaggio deve sicuramente essere maggiore di $\frac{1}{2^m}$. L'attacco quindi sarà banalmente il seguente:

```

A(SE)
   $m_0, m_1 \in \mathcal{M} \quad //: m_0 \neq m_1 \quad \wedge \quad |m_0| = |m_1|$ 
   $C \leftarrow Enc(m_0)$ 
   $\hat{C} \leftarrow Enc(m_1)$ 
  If  $(C == \hat{C})$  return 1
  else return 0
  
```

$Pr[Exp^{ind-cpa-1}(A) = 1] = 1$
 $Pr[Exp^{ind-cpa-0}(A) = 1] = 0$
 $Adv(A) = 1$

Come abbiamo fatto per la pseudo casualità, possiamo costruire un attaccante A che attacca il cifrario nel senso di indistinguibilità, sfruttando anche B, per cui varrà che:

$$Adv(B) \leq Adv(A) + \frac{1}{2^m}$$

```

AEnck(LR(·,·, b))
   $M_0, M_1 \leftarrow_R \{0, 1\}^m$ 
   $C \leftarrow Enc(LR(M_0, M_1, b))$ 
  Esegui B (con input C) come segue
  If B chiede il messaggio X
     $Y \leftarrow Enc_k(LR(X, X, b))$ 
    Return Y a B.
  Quando B si ferma e restituisce M
    If  $M == M_1$  return 1 else return 0
  
```

$b = 0$
 $C = Enc(m_0)$
 $C = Enc(m_1)$

Avremo che B quando chiede l'encryption di un messaggio passerà in input due volte la stessa stringa poiché l'oracolo è del tipo che cifra o solo il messaggio a sinistra o solo il messaggio a destra. Secondo questa definizione dell'attaccante A, avremo che:

$$Pr[Exp^{ind-cpa-1}(A) = 1] = Adv(B)$$

$$Pr[Exp^{ind-cpa-0}(A) = 1] = \frac{1}{2^m}$$

Un'ulteriore generalizzazione dell'indistinguibilità si può ottenere considerando un qualunque predicato e mostrando che, se si può estrarre, allora questo cifrario non è indistinguibile. Ragioniamo esattamente come prima, definiamo innanzitutto l'esperimento di B:

```

EspSEP (B)
  K ←R KeyGen; M ←R {0,1}m
  C ←R EncK(M)
  b ← BEncK(.)(C)
  If b=P(M) Return 1 else return 0

```

Il vantaggio di B sarà in questo caso $2Pr[Esp_{SE}^P(B) = 1] - 1$. Per cui creiamo un attaccante A che sfrutta B tale che il vantaggio di A sia:

$$Adv(B) = Adv(A)$$

```

AEncK(LR(·,·,b))
  M0, M1 ←R {0,1}m (Isb(M0 = b))
  C ← Enc(LR(M0, M1, b))
  Esegui B (con input C) come segue
    If B chiede il messaggio X
      Y ← EncK(LR(X, X, b))
      Return Y a B.
  Quando B si ferma e restituisce b'
    If b' = 1 return 1 else return 0.

```

Handwritten notes:
 - In the first line, b is written in blue.
 - In the second line, $(Isb(M_0 = b))$ is written in blue.
 - In the third line, $(LR(M_0, M_1, b))$ is written in blue.
 - In the fourth line, $(LR(X, X, b))$ is written in blue.
 - In the fifth line, b' is written in blue.
 - In the sixth line, $b' = 1$ is written in blue.
 - In the seventh line, b' is written in blue.
 - In the eighth line, b' is written in blue.
 - In the ninth line, b' is written in blue.
 - In the tenth line, b' is written in blue.
 - In the eleventh line, b' is written in blue.
 - In the twelfth line, b' is written in blue.
 - In the thirteenth line, b' is written in blue.
 - In the fourteenth line, b' is written in blue.
 - In the fifteenth line, b' is written in blue.
 - In the sixteenth line, b' is written in blue.
 - In the seventeenth line, b' is written in blue.
 - In the eighteenth line, b' is written in blue.
 - In the nineteenth line, b' is written in blue.
 - In the twentieth line, b' is written in blue.
 - In the twenty-first line, b' is written in blue.
 - In the twenty-second line, b' is written in blue.
 - In the twenty-third line, b' is written in blue.
 - In the twenty-fourth line, b' is written in blue.
 - In the twenty-fifth line, b' is written in blue.
 - In the twenty-sixth line, b' is written in blue.
 - In the twenty-seventh line, b' is written in blue.
 - In the twenty-eighth line, b' is written in blue.
 - In the twenty-ninth line, b' is written in blue.
 - In the thirtieth line, b' is written in blue.
 - In the thirty-first line, b' is written in blue.
 - In the thirty-second line, b' is written in blue.
 - In the thirty-third line, b' is written in blue.
 - In the thirty-fourth line, b' is written in blue.
 - In the thirty-fifth line, b' is written in blue.
 - In the thirty-sixth line, b' is written in blue.
 - In the thirty-seventh line, b' is written in blue.
 - In the thirty-eighth line, b' is written in blue.
 - In the thirty-ninth line, b' is written in blue.
 - In the fortieth line, b' is written in blue.
 - In the forty-first line, b' is written in blue.
 - In the forty-second line, b' is written in blue.
 - In the forty-third line, b' is written in blue.
 - In the forty-fourth line, b' is written in blue.
 - In the forty-fifth line, b' is written in blue.
 - In the forty-sixth line, b' is written in blue.
 - In the forty-seventh line, b' is written in blue.
 - In the forty-eighth line, b' is written in blue.
 - In the forty-ninth line, b' is written in blue.
 - In the fiftieth line, b' is written in blue.
 - In the fifty-first line, b' is written in blue.
 - In the fifty-second line, b' is written in blue.
 - In the fifty-third line, b' is written in blue.
 - In the fifty-fourth line, b' is written in blue.
 - In the fifty-fifth line, b' is written in blue.
 - In the fifty-sixth line, b' is written in blue.
 - In the fifty-seventh line, b' is written in blue.
 - In the fifty-eighth line, b' is written in blue.
 - In the fifty-ninth line, b' is written in blue.
 - In the sixtieth line, b' is written in blue.
 - In the sixty-first line, b' is written in blue.
 - In the sixty-second line, b' is written in blue.
 - In the sixty-third line, b' is written in blue.
 - In the sixty-fourth line, b' is written in blue.
 - In the sixty-fifth line, b' is written in blue.
 - In the sixty-sixth line, b' is written in blue.
 - In the sixty-seventh line, b' is written in blue.
 - In the sixty-eighth line, b' is written in blue.
 - In the sixty-ninth line, b' is written in blue.
 - In the seventieth line, b' is written in blue.
 - In the seventy-first line, b' is written in blue.
 - In the seventy-second line, b' is written in blue.
 - In the seventy-third line, b' is written in blue.
 - In the seventy-fourth line, b' is written in blue.
 - In the seventy-fifth line, b' is written in blue.
 - In the seventy-sixth line, b' is written in blue.
 - In the seventy-seventh line, b' is written in blue.
 - In the seventy-eighth line, b' is written in blue.
 - In the seventy-ninth line, b' is written in blue.
 - In the eightieth line, b' is written in blue.
 - In the eighty-first line, b' is written in blue.
 - In the eighty-second line, b' is written in blue.
 - In the eighty-third line, b' is written in blue.
 - In the eighty-fourth line, b' is written in blue.
 - In the eighty-fifth line, b' is written in blue.
 - In the eighty-sixth line, b' is written in blue.
 - In the eighty-seventh line, b' is written in blue.
 - In the eighty-eighth line, b' is written in blue.
 - In the eighty-ninth line, b' is written in blue.
 - In the ninetieth line, b' is written in blue.
 - In the hundredth line, b' is written in blue.

Come segnato, l'attaccante A sceglie random due messaggi per cui il predicato per ognuno ha un valore diverso.⁴ Dopodiché verrà eseguito B e darà la sua risposta cercando di capire se è stato cifrato il messaggio 0 o 1 in base al predicato che lui pensa possa avere quel testo cifrato mandato in input. Se B ritorna 1 vuol dire che pensa sia stato cifrato il messaggio 1 e che, quindi, il testo cifrato in input avesse il predicato 1, altrimenti si ragiona in modo speculare per il messaggio 0.

Cerchiamo di capire se il vantaggio è uguale:

$$\begin{aligned}
 Pr[Esp^{ind-cpa-1}(A) = 1] &= Pr[B \rightarrow 1] \\
 &= Pr[B \text{ indovina } P(M_1)] = Pr[Esp^P(B) = 1]
 \end{aligned}$$

$$\begin{aligned}
 Pr[Esp^{ind-cpa-0}(A) = 1] &= Pr[b' = 1] = Pr[B \rightarrow 1] \\
 &= Pr[B \text{ non indovina } P(M_0)] = 1 - Pr[B \text{ indovina } P(M_0)] = \\
 &= 1 - Pr[Esp^P(B) = 1]
 \end{aligned}$$

Poiché abbiamo che:

$$Adv^{ind-cpa}(A) = |Pr[Esp^{ind-cpa-1}(A) = 1] - Pr[Esp^{ind-cpa-0}(A) = 1]|$$

⁴Essendo un predicato il valore sarà o 0 o 1 e $P(M_1) = 1$ e $P(M_2) = 2$

utilizzando questa formula avremo che:

$$Pr[Esp^p(B) = 1] - (1 - Pr[Esp^p(B) = 1]) = 2Pr[Esp^p(B) = 1] - 1$$

Che è esattamente il vantaggio di B. Questo esperimento ci mostra che se non possiamo estrarre un predicato generico dal cifrario, allora non possiamo neanche attaccare il cifrario per indistinguibilità.

4.7 Sicurezza di CTRC e CTRC\$

Adesso che abbiamo dimostrato tutto ciò, possiamo dimostrare che i modi CTR sono sicuri. Per fare un breve ripasso, guardiamo i due modi di fare encryption:

Enc_k(M) Modo CTR\$ $F: \mathbb{K} \times \{0,1\}^\ell \rightarrow \{0,1\}^\ell$

```

m ← ⌈|M|/L⌉; R ←R {0,1}ℓ
Pad ← Fk(R+1)||Fk(R+2)||...||Fk(R+m)
Pad ← Primi |M| bits di Pad
C' ← M ⊕ Pad
C ← R||C'
return C

```

Enc_k(M) Modo CTRC

```

m ← ⌈|M|/L⌉;
If ctr+m ≥ 2ℓ return ⊥
Pad ← Fk(ctr+1)||Fk(ctr+2)||...||Fk(ctr+m)
Pad ← Primi |M| bits di Pad
C' ← M ⊕ Pad
ctr ← ctr+m
return ⟨ctr-m, C'⟩

```

Nonostante ci aspettiamo che i vantaggi siano uguali, questo non è propriamente vero. La differenza sta nel fatto che in CTR\$ prendiamo delle scelte random e, di conseguenza, siamo soggetti a un attacco basato su paradosso del compleanno mentre, invece, in CTRC ciò non accade perché non abbiamo il rischio di prendere una chiave uguale a una precedentemente scelta. La dimostrazione che faremo riguarderà solo CTRC. Se prendiamo A un avversario che attacca il cifrario per indistinguibilità facendo q domande per un totale di σ blocchi cifrati, esiste un avversario B che fa un attacco contro la sicurezza della PRF che fa σ domande e tale che:

$$Adv_{SE}^{ind-cpa}(A) \leq 2Adv_F^{prf}(B)$$

B^{prf}

```

b ← {0,1}
Run A : if A oracle ( $r_1, r_2$ ) do
        C ←  $O^{Enc}(r_b)$ 
    return C as A

A outputs b'
if b' == b output 1
else output 0

```

B sceglie un bit b che indicherà se verrà cifrata la stringa di sinistra o quella di destra facendo sì che B simuli in tutto e per tutto l'oracolo agli occhi di A. Dopodiché viene lanciato A e, se questo riesce ad indovinare quale stringa sta venendo cifrata, allora B risponde 1, altrimenti risponde 0.

$$Pr[Esp^{prf-1}(B) = 1] = Pr[Esp^{ind-cpa-cg}(A) = 1] \quad /B \text{ risponde 1 se } A \text{ risponde 1}$$

$$= \frac{1}{2} + \frac{1}{2} Adv^{ind-cpa}(A) \quad /Deriva dal fatto che $Adv^{ind-cpa}(A) = 2Pr[Esp^{ind-cpa-cg}(A) = 1] - 1$$$

$$Pr[Esp^{prf-0}(B) = 1] = Pr[Esp^{ind-cpa-cg}(A) = 1] = \frac{1}{2} + \frac{1}{2} Adv^{ind-cpa}(A) = \frac{1}{2}$$

/ $Adv^{ind-cpa}(A) = 0$ nel caso in cui siamo in prf-0 poiché la funzione è totalmente casuale

Dato ciò, avremo che:

$$Pr[Esp^{prf-1}(B) = 1] - Pr[Esp^{prf-0}(B) = 1] = \frac{1}{2} Adv^{ind-cpa}(A)$$

Da cui, segue che:

$$Adv(B) = \frac{1}{2} Adv(A)$$

Tutto questo ci indica che per rompere questo cifrario è necessario che la PRF non sia sicura. Per cui, se la PRF è sicura, allora CTRC è sicuro.

Per quanto riguarda CTR\$ non dimostreremo formalmente la cosa, tuttavia diremo che vale:

$$Adv_{SE}^{ind-cpa}(A) \leq Adv_F^{prf}(B) + \frac{0.5\sigma^2}{2^l}$$

Dove $\frac{0.5\sigma^2}{2^l}$ rappresenta la possibilità di un attacco basato sul paradosso del compleanno che viene introdotto dalla scelta casuale.

Un'importante cosa da tenere in considerazione sono i parametri che scegliamo nei cifrari come possono essere magari la grandezza dei blocchi, la grandezza della chiave ecc. Facciamo un esempio di scelta di questi parametri per quanto riguarda CTR.

Supponiamo di utilizzare:

$$F = AES(l = L = 128), q = 2^{30}$$

Ogni messaggio ha 2^{13} bit $\rightarrow 2^{36}$ blocchi

Posso utilizzare CTR\$ per cifrare q messaggi in modo sicuro? Per rispondere a questa domanda possiamo utilizzare l'avversario A che attacca CTR\$ e, dal teorema precedente, costruiamo B che, ricordiamo, attacca la prf del cifrario.⁵ Assumendo che AES sia sicuro, il vantaggio di B non può essere superiore a $\frac{\sigma^2}{2^{128}}$ cioè ciò che deriva dalla possibilità di un attacco basato sul paradosso del compleanno. Il teorema ci dice che:

$$Adv_{SE}^{ind-cpa}(A) \leq 2Adv_F^{prf}(B) \leq \frac{\sigma^2}{2^{128}} + \sigma^2 + 0.52^{128} = \frac{0.5 * 2^{72}}{2^{128}} \leq \frac{1}{2^{55}}$$

Questo ci dice che se cifriamo più di $\sigma = 2^{l/2}$ blocchi con CTR\$, non possiamo più dimostrare alcuna sicurezza.

4.8 Attacchi a crittotesto scelto

Fino a ora abbiamo considerato solo attacchi CPA,⁶ ma possiamo considerare un attacco ancora più forte che è quello a crittotesto scelto per cui forniamo all'attaccante un oracolo che può decifrare i messaggi. Se un cifrario è sicuro verso attacchi CCA, allora sarà sicuro anche nei confronti di CPA e sarà una definizione ancora più forte in un certo senso poiché diamo all'attaccante un'arma ben più forte rispetto a quella che già possiede.

$Esp_{SE}^{ind-cca-1}(A)$	$Esp_{SE}^{ind-cca-0}(A)$
$K \leftarrow_R \text{KeyGen}$	$K \leftarrow_R \text{KeyGen}$
$b \leftarrow A^{Enc_K(LR(.,.,1)), Dec_K(.)}$	$b \leftarrow A^{Enc_K(LR(.,.,0)), Dec_K(.)}$
If A imbrogia Return 0	If A imbrogia Return 0
else return b	else return b

$$Adv^{ind-cca}(A) = |\Pr[Esp_{SE}^{ind-cca-1}(A) = 1] - \Pr[Esp_{SE}^{ind-cca-0}(A) = 1]|$$

In figura possiamo vedere sia l'esperimento considerato che il vantaggio dell'attaccante. Notiamo che se l'attaccante "imbrogia", allora questo ritornerà un valore a caso. Diremo che un'attaccante imbrogia se questo chiede prima la cifratura di due messaggi e poi chiede la decifratura del crittotesto in output.

Proviamo ad attaccare CTR mediante un attacco a crittotesto scelto. Guardiamo il caso CTR\$ e consideriamo il seguente attaccante che sceglierà un messaggio da inviare e lo cifrerà. Dopodiché, modificherà opportunamente⁷ il crittotesto e lo decifrerà e, sulla base di questo, prenderà la sua decisione.

⁵Nello specifico AES

⁶Che è l'attacco a testo scelto

⁷La modifica che verrà provocata non è nota all'attaccante fino al momento della decifratura

$A^{\text{Enc}_k(LR(\cdot, \cdot, b)), \text{Dec}_k(\cdot)}$ un solo Blocco!
 $M_0 \leftarrow 0, M_1 \leftarrow 1$
 $(r, C) \leftarrow \text{Enc}_k(LR(M_0, M_1, b))$
 $C' \leftarrow C \oplus 1^\ell$
 $M \leftarrow \text{Dec}_k((r, C'))$
 If $M = M_0$ return 1 else return 0.

Durante queste operazioni, potremmo avere due flussi d'esecuzione:

1. $(r, C) \rightarrow C = F_k(r + 1) \oplus 1^L$
2. $(r, C) \rightarrow C = F_k(r + 1) \oplus 0^L = F_k(r + 1)$

E, al passo 3 avremo:

1. $c \oplus 1^L = F_k(r + 1) = F_k(r + 1) \oplus 0^L$ Che rappresenta esattamente ciò che abbiamo al passaggio 2 di prima.
2. $c \oplus 1^L = F_k(r + 1) = F_k(r + 1) \oplus 1^L$ che è esattamente ciò che abbiamo al passaggio 1 prima.

Dunque, una volta effettuata la decifratura del messaggio, vedremo che in ogni caso se inviamo M_1 otteniamo M_2 e viceversa. Notiamo che qualunque fosse stata la funzione di cifratura sottostante, non sarebbe cambiato nulla e ciò dimostra quanto gli attacchi CCA siano più forti rispetto agli attacchi CPA.

Capitolo 5

Funzioni Hash

Una funzione hash è una funzione che comprime e questo fa sì che un input di lunghezza arbitraria si trasformi in un output di lunghezza fissata. Un'importante caratteristica di queste funzioni sono le così dette **collisioni**. Definiamo una collisione per $H : D \rightarrow R$ una coppia tale che $H(x_1) = H(x_2)$ con $x_1 \neq x_2$.

Naturalmente se dovessimo avere che $|D| > |R|$ le collisioni sarebbero inevitabili per il teorema del **pigeon hole**. Diremo che H è **resistente alle collisioni** se è computazionalmente impossibile trovare delle collisioni. Considereremo una famiglia di funzioni

$$H : \{0, 1\}^k \times D \rightarrow \{0, 1\}^n$$

Per cui, fissata una chiave k , si definisce $h(x) = H_k(x)$.

Definiamo una funzione H **Universale o Quasi Universale** se il vantaggio $Adv^{cr0}(A) = Pr[Esp_H^{cr0}(A) = 1]$ è prossimo a 0 per tutti i possibili avversari esistenti. L'esperimento considerato è questo:

```
EspHcr0 (A)
(x1, x2) ←R A(); k ←R K;
If ((Hk(x1)=Hk(x2)) and (x1 ≠ x2) and (x1, x2 ∈ D))
  Return 1
else Return 0
```

Notiamo come in questo caso viene semplicemente scelta una coppia a caso di input e solo dopo si fissa una chiave.

Definiamo, invece, una funzione **Universale Unidirezionale** se il vantaggio $Adv^{cr1-kk}(A) = Pr[Esp_H^{cr1-kk}(A) = 1]$ per ogni possibile avversario computazionalmente limitato nel seguente esperimento è prossimo a 0. In buona sostanza, questo vantaggio è prossimo a 0 se non è possibile, a partire da un certo hash, estrarre computazionalmente tutti gli input che danno quell'hash.

```
EspHcr1-kk (A)
(x1, stato) ←R A(); k ←R K; x2 ←R A(k, stato);
If ((Hk(x1)=Hk(x2)) and (x1 ≠ x2) and (x1, x2 ∈ D))
  Return 1
else Return 0
```

Notiamo che l'esperimento è molto simile a quello precedente, tuttavia, la differenza sta nel fatto che prima scegliamo un input e, fissata la chiave, si sceglierà il secondo input. Abbiamo un elemento aggiuntivo in questo esperimento che viene rappresentato dalla variabile stato che non fa altro che salvare ciò che accade precedentemente.

Chiaramente una funzione universale unidirezionale è anche una funzione universale e ciò può essere dimostrato così: **Ipotesi:** H è una funzione universale unidirezionale.

Tesi: H è una funzione universale.

Quello che vogliamo dimostrare è che avremo che l'attaccante A che attacca la funzione con l'esperimento $cr1-kk$ limita superiormente l'attaccante B che attacca la funzione hash per universalità nell'esperimento $cr0$.

Immaginiamo di avere il seguente attaccante $A()$ che non fa altro che questo:

Algorithm 3 Attaccante A

A()
 1: $(x_1, x_2) \leftarrow B()$ ▷ Esegui la procedura $B()$
 2: $stato \leftarrow x_2$ ▷ Salva x_2 come stato
 3: **return** $(x_1, stato)$ ▷ Restituisci x_1 e il nuovo stato

Chiamata A(k, stato)
 4: $x_2 \leftarrow stato$ ▷ Carica lo stato precedente
 5: **return** x_2 ▷ Restituisci lo stato

Calcolando le probabilità vedremo che:

$$Adv(A) = Pr[Esp^{cr1-kk}(A) = 1] = Pr[Esp^{cr0}(B) = 1] = Adv(B)$$

Abbiamo quindi dimostrato che l'abilità di un avversario nel rompere la proprietà di universalità è limitata dall'abilità di un avversario nel rompere la proprietà di universalità unidirezionale.

Definiamo, infine, che una funzione è resistente alle collisioni se il vantaggio $Adv^{cr2-kk}(A) = Pr[Esp_H^{cr2-kk}(A) = 1]$ è prossimo a 0.

Exp_H^{cr2-kk} (A)
 $k \leftarrow_R K; (x_1, x_2) \leftarrow_R A(k);$ SCEGLIAMO LA CHIAVE
E POI FACCIAMO L'ATTACCO
If $((H_k(x_1) = H_k(x_2)) \text{ and } (x_1 \neq x_2) \text{ and } (x_1, x_2 \in D))$
 Return 1
else Return 0

Anche in questo caso possiamo dimostrare che una funzione resistente alle collisioni è anche universale e unidirezionale. Possiamo ancora una volta dimostrarlo così: **Ipotesi:** H è resistente alle collisioni.

Tesi: H è una funzione universale unidirezionale.

Supponiamo che H non sia universale unidirezionale ma che sia resistente alle collisioni. Allora esisterà un'attaccante B che attaccherà H come funzione universale unidirezionale e B sarà esattamente come era A prima.¹ Vediamo cosa succede:

Algorithm 4 Attaccante A

```

A(k)
1:  $RunB() \leftarrow (x_1, stato)$  ▷  $B$  attacca per u. u.
2:  $RunB(k, stato) \leftarrow x_2$  ▷ Runniamo nuovamente  $B$  con la chiave fissata
3: if  $H(x_1) = H(x_2)$  and  $x_1 \neq x_2$  then
4:   return  $(x_1, x_2)$ 
5: else
6:    $x'_1 \leftarrow Random()$  ▷ Genera  $x'_1$  casuale
7:    $x'_2 \leftarrow Random()$  ▷ Genera  $x'_2$  casuale
8:   return  $(x'_1, x'_2)$ 
9: end if

```

Calcoliamo anche in questo caso il vantaggio di A che sarà:

$$Adv(A) = Pr[Esp^{cr2-kk}(A) = 1] = Pr[Esp^{cr1-kk}(B) = 1] + A_{random} \geq Adv(B)$$

Ragioniamo adesso sulle chiavi, spesso noi facciamo delle affermazioni considerando una chiave non fissata, tuttavia è possibile che noi troviamo degli attacchi specifici per una singola chiave e non per tutte. Questa è una cosa che nei fatti potrebbe realmente accadere, come vedremo tra poco, tutte le funzioni hash utilizzano una chiave fissata o possono addirittura non utilizzarne una.

5.1 Applicazioni di funzioni Hash

Cerchiamo, prima di tutto, di capire a cosa servono effettivamente le funzioni Hash e perché sono così importanti al giorno d'oggi. Di fatto, senza funzioni Hash, non conosceremmo la rete per quello che è oggi e vediamo perché.

Una delle applicazioni più conosciute è, banalmente, la verifica della password che avviene ogni qualvolta effettuiamo il login su un qualunque sito web. Nome utente e password sono salvati su un database che potrebbe essere soggetto ad attacchi per cui, se la password venissero salvate in chiaro, un qualunque hacker avrebbe nome utente e password di tutti gli utenti registrati. Utilizzando le funzioni hash ciò non accade poiché, salvando l'hash della password, anche se venisse compromesso il database, non ci sarebbe modo di ricavare la password originale.

Un'altra cosa che può essere effettuata è la verifica d'integrità di grossi file. Immaginando di voler verificare che due file di grosse dimensioni siano uguali, confrontarli bit a bit è sicuramente un'operazione dispendiosa. Una cosa molto più comoda è generare gli hash dei due file e confrontare solo quelli che, quindi, avranno dimensione minore rispetto a quella originale.

Allo stesso modo si può agire quando si scaricano dei file da qualche sito non ufficiale controllando se l'eseguibile appena scaricato "''''''PAGANDO''''''" ha lo stesso hash dell'eseguibile ufficiale.

¹L'attaccante dell'esperimento $cr1-kk$ che poteva prendere come parametri nulla e poi k , stato

5.2 SHA-1 (Secure Hash Algorithm 1)

SHA-1 è una funzione di hashing tale che:

$$SHA1 : \{0, 1\}^{<2^{64}} \rightarrow \{0, 1\}^{160}$$

Questa fu proposta nel 1995 e rimase lo standard fino al 2006 anche se, in rarissimi casi, è tutt'oggi utilizzata. L'idea alla base di questo algoritmo è quello di utilizzare una funzione di compressione che va da $512 + l$ bit a l bit soltanto. Questa idea viene comunemente chiamata **Merkle-Damgard**. Questa si compone di diverse procedure che comprendono il modificare l'input attraverso una procedura chiamata **SHAPAD** che trasforma la lunghezza del messaggio in un multiplo di 512. Dopodiché, viene iterata la funzione di compressione **shf1**.

```

SHA1(M)      // |M| < 264
    V ← SHF1(5A827999||6ED9EBA1||8F1BBCDC||CA62C1D6, M)
    return V

SHF1(K, M)    // |K| = 128 e |M| < 264
    y ← shapad(M)
    Sia y M1||M2||...||Mn (|Mi| = 512)
    V ← 67452301||EFCDA8B9||98BADCFE||10325476||C3D2E1F0
    for i = 1, ..., n
        V ← shf1(K, Mi||V)
    return V

shapad(M)     // |M| < 264
    d ← (447 - |M|) mod 512
    Sia ℓ la rapp. binaria (64 bit) di |M|
    y ← M||1||0d||ℓ
    return y

```

Handwritten notes:

- CHIAVE F.CSH (pointing to the key in SHF1)
- UNO DEI MULTIPLI DI 512 (pointing to the padding in shapad)
- 512 + 160 = 160 (pointing to the output of shf1)
- MAX 2⁶⁴ BIT E LA RAPPRESENTAZIONE IN 64 BIT (pointing to the length ℓ)
- |y| = |M| + 1 + d + 64 = 447 + d + 64 + t 512

Analizzando le procedure a grandi linee:

- **SHA1(M)**: Funzione principale che si occupa di fare l'hashing.
- **SHF1(K,M)**: Questa funzione si occupa di fare il padding del messaggio, suddividerlo nei vari blocchi e, a partire dalla chiave K che è **fissa** per tutte le applicazioni di SHA1, inizia un processo che si occupa di concatenare il blocco del messaggio di 512 bit al V precedente di 160 bit per poi comprimerlo in 160 bit nuovamente in maniera da poter continuare nuovamente il processo. Ciò che verrà restituito sarà V di 160 bit che sarà l'hash del messaggio.
- **SHAPAD(M)**: Shapad è la funzione che viene richiamata da SHF1 per effettuare il padding del messaggio e serve, per l'appunto a far sì che il messaggio abbia una lunghezza che è un multiplo di 512.

5.3 Attacchi alle funzioni Hash

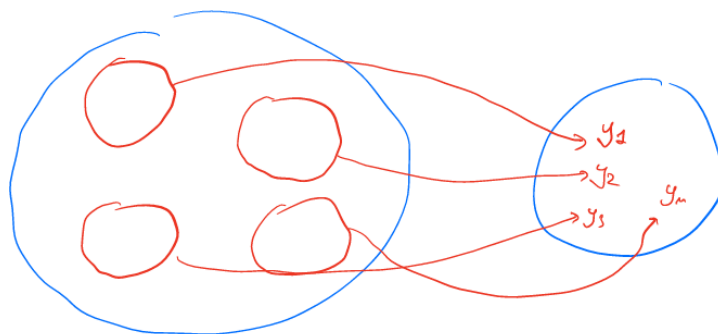
Analizziamo i possibili attacchi alla proprietà di resistenza alle collisioni che è quella più utile per quanto riguarda i nostri scopi.

L'attacco più banale che ci può venire in mente è quello di forza bruta per cui scegliamo un input a caso x_1 e calcoliamo il suo hash $H(x_1)$. Dopodiché, proviamo tutti i possibili x_2 fino a quando non abbiamo che $H(x_1) = H(x_2)$. Questo attacco avrà nel caso peggio 2^{160} iterazioni poiché questo è il numero dei possibili output diversi che si possono avere.

Ripensando al paradosso del compleanno ci possiamo però immaginare uno scenario migliore rispetto ad un semplice attacco di forza bruta. Possiamo prendere due input q a caso e calcolare i rispettivi hash per cui il paradosso del compleanno sembrerebbe garantirci una complessità $O(\sqrt{|R|})$.

Avversario A	Attacco del Compleanno
<pre> for i=1,...,q do $x_i \leftarrow_R D$; $y_i \leftarrow H_k(x_i)$; if $\exists i,j (i \neq j \wedge y_i = y_j \wedge x_i \neq x_j)$ COLL $\leftarrow$ True </pre>	<pre> for i=1,...,q do $y_i \leftarrow_R \{0,1\}^n$; if $\exists i,j (i \neq j \wedge y_i = y_j)$ COLL $\leftarrow$ True </pre>
$\Pr[\text{COLL}] = ?$ <i>y non è distribuito in maniera uniforme</i>	$\Pr[\text{COLL}] = q^2 / 2^{n+1}$

Notiamo però come la differenza stia proprio nel fatto che l'hash calcolato in uno specifico punto non è casuale e, inoltre, la funzione potrebbe non essere distribuita in maniera regolare. Per cui questo non sembra garantirci nulla riguardo alla possibilità di trovare la collisione. Quello che facciamo, d'altronde, non è altro che cercare una cosa di questo tipo:



Immaginando che l'insieme più grande sia il dominio degli input, questo sarà diviso in vari sottoinsiemi che corrisponderanno a tutti gli specifici x_i che danno lo stesso Hash e noi cercheremo di prendere nel pratico due elementi appartenenti allo stesso sottoinsieme. Per far sì che questa operazione sia difficile, è necessario avere quanti più sottoinsiemi possibili cosicché gli elementi che mappano lo stesso output

siano pochi e ciò non dipende dalla cardinalità dell'insieme di input, bensì dalla cardinalità dell'insieme di output. Banalmente, maggiori sono gli output possibili, meglio si distribuiranno i nostri input.

L'attacco risulta ancora più efficace se la funzione non è regolare poiché se si creano sottoinsiemi più grossi di altri all'interno del dominio, sarà più facile prendere due elementi appartenenti allo stesso insieme.

Teorema

Siano:

- $h : K \times \{0, 1\}^{b+v} \rightarrow \{0, 1\}^v$ una famiglia di funzioni.
- $H : K \times D \rightarrow \{0, 1\}^v$ costruita come descritto.
- A_H un avversario che trova collisioni in H .

Allora possiamo costruire A_h che trova collisioni per h tale che:

$$Adv_H^{cr2-kk}(A_H) \leq Adv_h^{cr2-kk}(A_h)$$

Sostanzialmente ciò ci dice che se h è resistente alle collisioni, allora ciò varrà anche per H .

5.4 Sponge Construction

Questo paradigma è alternativo a Merkle Damgaard ed è un semplice processo iterativo basato su una specifica permutazione:²

$$P : \{0, 1\}^n \rightarrow \{0, 1\}^n$$

Questa operazione contiene due parametri fondamentali r detto rate, c detto capacity : $r + c = n$ dove n rappresenta la lunghezza della stringa. Troviamo, inoltre, v che rappresenta la dimensione dell'output e $L - max$ che rappresenta la massima dimensione dei messaggi che è, in genere, $\{0, 1\}^{<L+1}$.

Questo algoritmo si compone di due fasi principali:

1. **Absorbing stage:** Durante questa fase aggiungiamo il padding³ ad M per far sì che questo diventi multiplo di r , inizializziamo la stringa h a una stringa null e, a ogni blocco ottenuto dalla divisione di M , aggiungiamo c 0 in maniera tale da avere blocchi di n bit per poi, infine, applicare la permutazione allo xor tra h e il messaggio con gli 0 concatenati.

²Come dicevamo, non abbiamo chiave

³L'unica cosa necessaria per il padding è che sia iniettivo. Cioè, vogliamo che a messaggi diversi, tramite il padding, si arrivi a stringhe diverse

```
// Absorbing stage
 $m \leftarrow \text{Pad}(M)$ 
Sia  $m = m_1 || \dots || m_s$  ( $|m_i| = r$ )
 $h \leftarrow 0^n$ 
for  $i = 1$  to  $s$  do
     $m'_i \leftarrow m_i || 0^c \in \{0, 1\}^n$ 
     $h \leftarrow P(h \oplus m'_i)$ 
```

2. **Squeezing stage:** Una volta fatto ciò, continuiamo ad applicare la permutazione ad h e di questo prendiamo via via i primi r bit che rappresenteranno z_i . Il nostro output finale saranno i primi v bit della concatenazione di tutti gli z_i .⁴

```
// Squeezing stage
 $z_1 \leftarrow \text{Primi } r \text{ bit di } h$ 
for  $i = 2$  to  $\lceil v/r \rceil$  do
     $h \leftarrow P(h)$ 
     $z_i \leftarrow \text{Primi } r \text{ bit di } h$ 
output i primi  $v$  bit di  $(z_1 || z_2 || \dots || z_{\lceil v/r \rceil})$ 
```

Lo standard NIST specifica una famiglia di funzioni Sponge-based basate sulla permutazione Keccak[c](m, v) dove c rappresenta la capacità e, generalmente, si utilizza il padding 10, m il messaggio e v la lunghezza dell'output.

⁴I bit sprecati saranno solo nell'ultimo z

Capitolo 6

Message Authentication

Fino a ora abbiamo parlato di sicurezza in termini di privacy. Tuttavia, soprattutto negli ultimi anni, è parecchio importante garantire autenticità poiché spesso la privacy non è necessaria.

Il nostro obiettivo è quello di far sì che il mittente S , che vuole inviare un messaggio M al destinatario R , possa rendersi conto di eventuali corruzioni da parte di esterni; In questo modo verrebbero garantite autenticità e integrità del messaggio. Partiremo dal presupposto che S e R hanno una chiave segreta k in comune. L'autentica di un messaggio M produce un messaggio M' che fa' sì che se R vede qualcosa di diverso da M' si renda conto che il messaggio non è autentico.

6.1 Message Authentication Codes

Spesso M' è proprio il messaggio M concatenato ad una stringa di lunghezza fissa detta **tag** che permetterà l'autenticazione del messaggio. Se l'autenticazione non va' a buon fine, il comportamento di R potrebbe variare in base al contesto; si potrebbero inviare messaggi di errore, si potrebbe interrompere la comunicazione ecc.

L'avversario parteciperà alla comunicazione e supporremo che questo abbia il pieno controllo di essa.

A questo punto è lecito chiedersi se possa essere utile utilizzare gli strumenti già visti nei capitoli precedenti per ottenere autenticità. Banalmente, se S vuole mandare M a R , S calcola l'encryption del messaggio e poi r , una volta ricevuto lo decifrerà e, se il risultato ha "senso", allora lo dichiarerà come autentico. Intuitivamente questa cosa ha senso, se A non conosce M , non può nemmeno modificarlo in modo corretto. Tuttavia potremmo essere in grado di modificare il messaggio senza conoscerlo parzialmente o interamente. Ad esempio, se noi riuscissimo a cambiare c per far sì che 100 diventi 900, avremmo modificato il messaggio ma questo rimarrebbe di senso compiuto.

Qui il problema non è quindi il cifrario utilizzato, semplicemente questi meccanismi sono stati progettati per scopi diversi, e quindi, su misura per specifiche diverse a quelle che stiamo cercando di ottenere adesso. È più opportuno nel nostro caso, creare su misura gli strumenti che ci saranno di aiuto.

6.2 MAC

Per definire il MAC necessitiamo di questa serie di algoritmi:

- **KeyGen**: questo algoritmo si occupa di restituire una stringa casuale.
- **MAC**: Prendiamo in input una chiave k , un messaggio $M \in \{0, 1\}^*$ e restituiamo un $tag \in \{0, 1\} \cup Err$
- **VF**: Questo algoritmo è quello di verifica e non fa' altro che prendere in input k , M e un tag e restituisce 1 se la verifica va' a buon fine e 0 altrimenti.

Imponiamo che i verificatori siano deterministici poiché gestire destinatari a stati potrebbe diventare problematico a causa di problemi di sincronizzazione o di sicurezza della variabile stati. Eppure, non c'è dubbio che questi siano utili poiché, in certe situazioni, potrebbero prevenire attacchi di replay. Nonostante ciò questi non verranno trattati in questo corso.

Considereremo, invece, sistemi **deterministici** che, se nei cifrari sembrano creare problemi, vedremo come qui saranno utili. Se il MAC è deterministico, avremo che l'algoritmo di verifica sarà sempre lo stesso, cioè:

```

VFk( $M, tag$ )
   $tag' \leftarrow MAC_k(M)$ 
  if  $tag' = tag \wedge tag \neq \perp$  return 1 else return 0.

```

6.3 Modello di attaccante

Dato che vogliamo dare una precisa definizione di sicurezza per quanto riguarda il MAC, possiamo lavorare come prima definendo innanzitutto un obiettivo e, successivamente, un attaccante adeguato.

Il nostro obiettivo sarà, quindi, quello di rilevare ogni tentativo di modifica da parte di A. E, per fare ciò, A cercherà di produrre dei **falsi** tali che $VF_k(M, tag) = 1$. Per far sì che il nostro approccio sia il più generico possibile, non ci limiteremo a considerare solo messaggi che abbiano un senso, bensì qualunque tipo di messaggio poiché, se riusciamo a difenderci contro qualunque messaggio sia anche privo di senso, potremo farlo anche da messaggi con un senso.

L'attaccante avrà in genere accesso a coppie (M, tag) validi per cui lui potrà anche chiedere il tag di specifici messaggi m ; chiameremo questo tipo di attacco *CMA*, cioè, chosen message attack. Possiamo anche definire il seguente esperimento a cui parteciperà l'attaccante:¹

¹UF sta per unforgiability

MA: (KeyGen,MAC,VF)

```

EspMAuf-cma (A)
  k ←R KeyGen(); Esegui AMACk(.),VFk(.,.)
  if A fa una richiesta di verifica (M,tag) tale che
    - L'oracolo di verifica risponde 1
    - A non aveva richiesto M all'oracolo MAC
    Return 1
  else Return 0

```

Qui faremo attenzione a due oracoli:

- **MAC**: che prenderà un messaggio M e restituirà il tag associato.
- **VF**: che si occuperà di verificare se una coppia (messaggio, tag) sia corretta.

E diremo che un MA è sicuro se il vantaggio $Adv^{uf-cma}(A)$ è prossimo a 0:

$$Adv^{uf-cma}(A) = Pr[Exp_{MA}^{uf-cma}(A) = 1]$$

È importante separare per i nostri scopi q_s che rappresenta il numero di domande all'oracolo MAC e q_v che rappresenta il numero di domande all'oracolo di verifica. Questo ci tornerà comodo per l'analisi che faremo successivamente.

Innanzitutto facciamo alcuni esempi di MAC non sicuri e vediamo quale è il problema. Ricordiamoci bene che il nostro obiettivo sarà quello di prevedere il MAC di un messaggio diverso da quelli che abbiamo già chiesto.

MA: (KeyGen,MAC,VF)

```

EspMAuf-cma (A)
  k ←R KeyGen(); Esegui AMACk(.),VFk(.,.)
  if A fa una richiesta di verifica (M,tag) tale che
    - L'oracolo di verifica risponde 1
    - A non aveva richiesto M all'oracolo MAC
    Return 1
  else Return 0

```

Per capire se è sicuro, possiamo ragionare allo stesso modo di come abbiamo fatto per le funzioni pseudo-random. Vediamo di costruire un attaccante che è in grado di prevedere il tag di un messaggio.

$A(m)$
 $X \leftarrow \{0,1\}^L$
 $n \leftarrow X \parallel X$
 $tag \leftarrow 0^L$
 $O^{VER}(m, tag)$

$Ver_K(m, tag)$
 $tag' \leftarrow MAC_K(m)$
 $if (tag == tag') \wedge (tag \neq \perp)$
 output 1
 else output 0

Oppure ancora:

$\exists F: \{0,1\}^k \times \{0,1\}^{L+L} \rightarrow \{0,1\}^L \quad M = \{0,1\}^{2L}$

$MAC_K(M)$
 $n = n_1 \parallel n_2 \quad n_1, n_2 \in \{0,1\}^L$
 $tag \leftarrow F_K(n_1 \parallel 0) \parallel F_K(n_2 \parallel 1)$
 output tag

$A(M)$
 $x, y, z, w \in \{0,1\}^L \quad x \neq y \neq z \neq w$
 $n_1 \leftarrow x \parallel y; \quad n_2 \leftarrow z \parallel w$
 $tag_1 \leftarrow O^{MAC}(n_1); \quad tag_2 \leftarrow O^{MAC}(n_2)$
 $n_3 \leftarrow x \parallel w \quad tag_3 \leftarrow [tag_1]_1 \parallel [tag_2]_2$
 $O^{VER}(n_3, tag_3)$

$tag_1 = \alpha \parallel \beta$
 $\uparrow \quad \uparrow$
 $tag_2 = \gamma \parallel \delta$
 $tag_3 = \alpha \parallel \delta$

Infine:

$MAC_K(M)$
 $if |M| \bmod C \neq 0 \quad output \perp$
 Sia $M = M_1 \parallel M_2 \parallel \dots \parallel M_m$
 For $i = 1 \dots m \quad y_i \leftarrow F_K(m_i)$
 tag $\leftarrow y_1 \parallel y_2 \parallel \dots \parallel y_m$

$A(M_K)$
 $m_i \leftarrow 0^L \parallel 1^L$
 $tag_i \leftarrow O^{MAC}(m_i)$
 $tag_1 = tag_n \parallel tag_{12}$
 $m_2 \leftarrow 1^L \parallel 0^L; \quad tag_2 \leftarrow tag_{12} \parallel tag_{11}$
 $O^{VER}(M_2, tag_2)$

6.4 CBC-MAC

Dagli esempi fatti, quello che sembra è che le prf non siano buone per costruire dei MAC sicuri, tuttavia, in realtà questo approccio è sicuro. Per fare ciò, banalmente si lavora a questo modo:

$F: K \times D \rightarrow \{0,1\}^\tau$ fam. di funz.

KeyGen $k \leftarrow_R K$ return k	MAC_k(M) If $(M \notin D)$ return $\perp$ $\text{Tag} \leftarrow F_k(M)$ return Tag
---	---

Da qui abbiamo un teorema per cui:
 Sia $F : K \times D \rightarrow \{0,1\}^\tau$ una famiglia di funzioni, sia $MA = (\text{KeyGen}, \text{Mac})$ il metodo appena descritto. Preso A un avversario che attacca MA facendo $q_s + q_v$ domande, esiste un avversario B che attacca la prf che fa $q_s + q_v$ domande tale che:

$$Adv_{MA}^{uf-cma}(A) \leq Adv_F^{prf}(B) + \frac{q_v}{2^\tau}$$

Ciò vuole sostanzialmente dirci che possiamo rompere MA solo se rompiamo la prf sottostante. Costruiamoli:

```

B(F)
d = 0; S = void
Run A(MA) if A chiede M all'oracolo MAC:
    B domanda all'oracolo prf M e invia F(M) ad A
    S = S U {M}
if A chiede (M, tag) all'oracolo VER:
    if (f(M) == tag)
        rispondi 1 ad A
        if M not in S
            d = 1
    else
        rispondi 0 ad A
output d
  
```

Calcoliamo i vantaggi:

$$\begin{aligned}
 Adv(B) &= Pr[Esp^{prf-1}(B) = 1] - Pr[Esp^{prf-0}(B) = 1] \\
 Pr[Esp^{prf-1}(B) = 1] &= Adv(A) \\
 Pr[Esp^{prf-0}(B) = 1] &= Pr[\tau_1 \vee \tau_2 \vee \dots \vee \tau_{q_v}] \leq \\
 \sum_i Pr[\tau_i] &= \frac{q_v}{2^\tau}
 \end{aligned}$$

Il problema di questo tipo di utilizzi è il fatto che la cifratura di un messaggio genera un tag della stessa dimensione del messaggio per cui abbiamo un consumo di banda importante.

Un metodo più efficiente è quello chiamato **CBC-MAC** per cui a partire da un cifrario a blocchi $E : \{0, 1\}^k \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ si costruisce il tag nel seguente modo:

```

MACk(M)
  if  $M \notin \mathcal{M}$  return  $\perp$ 
    //  $\mathcal{M}$  spazio dei messaggi
  Sia  $M = M[1] \dots M[m]$ 
    //  $|M[i]| = n$ 
   $C[0] \leftarrow 0^n$   $C[0] \leftarrow 0^m$ 
  for  $i = 1, \dots, m$ 
     $C[i] \leftarrow E_k(C[i-1] \oplus M[i])$ 
  return  $C[m]$  // Tag =  $C[m]$ 
    ↪ SOLO L'ULTIMO ELEMENTO

```

Teorema

Sia un cifrario a blocchi $E : \{0, 1\}^k \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ e sia $\{0, 1\}^{nm}$ lo spazio dei messaggi. Preso un avversario A contro CBC-MAC che fa q richieste di autentica e 1 richiesta di verifica, esisterà un avversario B che attacca E come PRP² che fa q+1 domande tale che:

$$Adv_{MAC}^{uf-cma}(A) \leq Adv_E^{prp-cpa}(B) + \frac{m^2 q^2}{2^{n-1}}$$

In questo teorema, assumiamo che tutti i messaggi abbiano la stessa taglia e questo è necessario per la validità del teorema. Supponiamo che si possano prendere messaggi di lunghezza diversa:

$$M_1 = 0^m \quad tag_1 = E_k(0^m)$$

$$M_2 = x \in \{0, 1\}^m \quad tag_2 = E_k(x)$$

$$M_3 = M_2 || tag_2 = x || E_k(x)$$

$$\hookrightarrow tag_3 = C[1] = E_k(0^m \oplus x) = E_k(x)$$

$$C[2] = E_k(C[1] \oplus E(x)) = E_k(0^m) = \boxed{tag_1}$$

Similmente, possiamo anche dimostrare che CBC-MAC randomizzato non è sicuro:

²Permutazione random dato che stiamo lavorando con un cifrario a blocchi

CBC-MAC RANDOMIZZATO

 $\text{Mac}_K(M)$ IF $M \notin m$ OUTPUT $\perp$ $M = M[1] \parallel \dots \parallel M[m]$ $R = C[0] \leftarrow \{0,1\}^m$ FOR $i=1$ TO m $C[i] \leftarrow E_K(C[i-1] \oplus M[i])$ RETURN $(R, C[m])$ $\text{Ver}_K(M, (R, C))$ $M = M[1] \parallel \dots \parallel M[m]$ $C[0] \leftarrow R$ FOR $i=1$ TO m $C[i] \leftarrow E_K(C[i-1] \oplus M[i])$ IF $(C[m] = C)$ RETURN $\perp$ ELSE RETURN 0

Il vero e proprio problema di questo metodo è che R , essendo randomizzato, deve essere obbligatoriamente rilasciato insieme al tag vero e proprio, dunque:

 $M_1 \rightarrow (R_1, C_1)$ $M_2 \parallel R_1 \quad M_2 \oplus R_1 = R_1 \oplus M_1 \quad R_2 = R_1 \oplus M_1 \oplus M_2$ FACCIAMO LA VERIFICA DI $M_2 \parallel R_2$ $C[0] \leftarrow R_2$ $C[1] = E_K(R_2 \oplus M_2) = E_K(R_1 \oplus M_1) = C_1$

Abbiamo che il Mac di $M_2 \parallel R_2$ è C_1 del primo messaggio. Cioè, siamo riusciti ad ottenere uno specifico Mac per $M_2 \parallel R_2$

Il fatto che CBC-MAC sia deterministico è fondamentale per la sua sicurezza, poiché grazie a ciò, non è possibile replicare l'attacco precedente:

- Quando l'attaccante riceve il tag per M_1 , ottiene $\text{tag}_1 = E_k(0^n \oplus M_1)$.
- Se ora vuole falsificare un tag per M_2 , dovrebbe calcolare $\text{tag}_2 = E_k(0^n \oplus M_2)$.
- Non può usare la tecnica di manipolazione dell'IV (perché l'IV è fisso e non può controllarlo) e, senza conoscere la chiave k , non può calcolare il risultato corretto.

Se volessimo ottenere CBC-MAC per messaggi a lunghezza variabile, avremo principalmente tre tipi di approcci:

1. Utilizzare **chiavi indipendenti per messaggi di lunghezza diversa**. Questo approccio è banale, per ottenere chiavi diverse in base alla lunghezza potremmo fare a questo modo:

$$k_l = F_k(l)$$

$$\text{tag} = \text{CBC-MAC}(k_l, m)$$

2. Possiamo **prependere** a m la sua lunghezza così da non avere la necessità di fare un ulteriore calcolo dopo. Supponendo di voler effettuare l'attacco precedente, questo non sarebbe più fattibile poiché due messaggi di lunghezza diversa hanno due blocchi iniziali diversi e, quindi, nulla in comune.

3. Possiamo utilizzare due **chiavi indipendenti** k_1, k_2 per poi fare:

$$\begin{aligned} t &= CBC - MAC(k_1, m) \\ tag &= CBC - MAC(k_2, t) \end{aligned}$$

Anche in questo caso l'attacco non sarebbe replicabile perché le due iterazioni sono separate. Una cosa da segnalare è che la seconda iterazione di CBC è comunque fatta con un solo blocco e, quindi, molto più veloce della prima.

6.5 MAC da HASH (HMAC)

Nonostante tutta la fatica fatta per ottenere CBC-MAC, questo è estremamente lento e, in pratica, quasi mai utilizzato. Al contrario, molto famosi sono i MAC ottenuti da funzioni hash costruite secondo la metodologia Markle-Damgard che hanno, come unico svantaggio, il dover assumere che la funzione di compressione abbia proprietà di pseudo casualità. Facciamo le seguenti ipotesi:

- H, h restituiscono output di n bit.
- Se m è in $\{0, 1\}^b$, $H(m)$ richiede una sola esecuzione di h .
- **IV, opad, ipad** sono costanti di lunghezza b .

Vediamo quindi il funzionamento di **HMAC**:

KeyGen

- $s \leftarrow \text{HASH-KeyGen}$.
- Sceglie una stringa casuale k di b bit.
 - b lunghezza del blocco

$\text{MAC}((s,k),m) \quad |m|=L$

$$\text{tag} \leftarrow H_s((k \oplus \text{opad}) || H_s(k \oplus \text{ipad} || m))$$

Questa funzione è molto efficiente, facile da implementare e molto più sicura di tutte le altre funzioni non sicure adottate in commercio prima di questa standardizzazione.

Immaginiamo adesso di voler ottenere cifratura e autenticazione, le sole due cose possibili sono:

I Soluzione

$\text{tag} \leftarrow \text{Mac}(K_1, m)$
 $c \leftarrow \text{Enc}(K_2, m)$
 Output (tag, c)

$\text{MDec}(K_1, K_2, (c, \text{tag}))$
 $m \leftarrow \text{Dec}(K_2, c)$
 $b \leftarrow \text{VF}(K_1, m, \text{tag})$
 if $b == 1$ output m
 else output $\perp$

II Soluzione

$c \leftarrow \text{Enc}(K_2, m)$
 $\text{tag} \leftarrow \text{Mac}(K_1, c)$
 Output (tag, c)

$\text{MDec}(K_1, K_2, (c, \text{tag}))$
 $b \leftarrow \text{VF}(K_1, c, \text{tag})$
 if $b == 0$ output $\perp$
 else $m \leftarrow \text{Dec}(K_2, c)$
 output m

Quindi, o facciamo prima la cifratura e poi calcoliamo il MAC oppure il contrario. Per quanto le cose sembrino apparentemente identiche, in realtà pensandoci bene non è sicuro effettuare prima il mac e poi la cifratura poiché inviando due volte lo stesso messaggio, il mac sarebbe lo stesso ma con la cifratura diversa. Facendo il contrario, essendo che la cifratura del messaggio cambia, cambia anche il suo mac, rendendo il secondo approccio sicuro rispetto all'altro.

Capitolo 7

Primitive Asimmetriche

La Crittografia Simmetrica parte dal presupposto che i due interlocutori possiedano una chiave condivisa segreta. Tuttavia, questo non è facilmente realizzabile poiché servirebbe un canale sicuro e, come già ben sappiamo, non esiste. L'idea alla base della crittografia asimmetrica è quella di avere due chiavi una **pubblica** per cifrare e l'altra **privata** per decifrare. Una proprietà fondamentale che devono possedere queste due chiavi è che da una non si possa ricavare l'altra in maniera efficiente e viceversa. Dunque, ora più che mai, è importante fare riferimento alla **teoria della complessità** per renderci conto di quanto l'avversario¹ possa o meno effettuare l'attacco. Ci baseremo sull'utilizzare algoritmi che sfruttano operazioni efficienti ma che sia intrattabile effettuare l'operazione inverse.²

Informalmente, si tratta di funzioni facili da calcolare ma difficili da invertire. Per quanto riguarda la facilità di calcolare la funzione, vuol dire che:

$$\exists A : \forall x \in D, A(x, f) = f(x)$$

Mentre, per il fatto che debba essere difficile invertire la funzione, vuol dire che:

$$\nexists A \text{ polinomiale} : A(y, f) = z, f(z) = y$$

Per essere più formali:

- Diciamo che μ è **trascurabile** se $\forall c \geq 0 \quad \exists k_c : \mu(k) \leq k^{-c}$.
- Una funzione $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ è **unidirezionale** se:
 1. Esiste un algoritmo efficiente (PPT) che su input x calcola $f(x)$.
 2. Per ogni algoritmo A , esiste una funzione μ_A tale che, per k sufficientemente grande:

$$Pr[f(z) = y : x \in_R \{0, 1\}^k; f(x) = y; A(k, y) = z] \leq \mu_A(k)$$

cioè vuol dire che la probabilità di invertire la funzione è trascurabile.

¹Ricordiamoci che deve sempre essere limitato

²In assenza di chiave privata

Per la definizione appena data, A non è ancora capace di invertire la funzione, o meglio, nessuno lo è. Inoltre, non è ancora stato provato che esistano delle funzioni unidirezionali.

Supponiamo adesso di avere una funzione **unidirezionale trapdoor**, ossia una funzione unidirezionale per cui se posseduta una certa informazione, allora è facile da invertire. Supponendo di prendere delle permutazioni trapdoor, allora sarebbe possibile costruire dei cifrari in modo piuttosto semplice:

Bob pubblica una permutazione unidirezionale trapdoor f (e mantiene segreta la trapdoor sk).

Alice cifra m nel seguente modo:

$$c = f(m)$$

Bob decifra c calcolando $f^{-1}(c) = m$

Attenzione, a questo punto la funzione non è ancora randomizzata, tuttavia a questo ci arriveremo più avanti.

A questo punto ci vengono in aiuto alcuni problemi detti **NP-completi**, questa particolare categoria di problemi possiede la caratteristica per cui, se la soluzione è data, questa è facilmente intuibile, tuttavia calcolarla da 0 è un problema intrattabile.

7.1 Un po' di Algebra

Prima di procedere con la spiegazione, è bene fare un po' di ripasso riguardo ad alcuni concetti algebrici che, però, non saranno oggetto d'esame. Definiamo $Z_N = \{x : 0 \leq x < N\}$ e definiamo l'operazione somma come $a, b \in N, a + b \bmod N$ che gode di proprietà associativa e presenta l'elemento neutro per cui $a + 0 = a$ e l'inverso $a + a' = 0 \bmod N$. Questo è, quindi, un gruppo additivo.

Ora, dato $Z'_N = \{0 < i < N : \gcd(i, N) = 1\}$ dove "gcd" rappresenta il massimo comune divisore, per cui, Z'_N è l'insieme dei numeri coprimi tra i e N . Ad esempio:

$$Z'_{15} = \{1, 2, 4, 7, 8, 11, 13, 14\}$$

Possiamo qui definire l'operazione moltiplicazione modulo N come $a, b \in Z'_N, a * b = c \bmod N$. Questa operazione si può dimostrare rimanere sempre nello stesso insieme. Diciamo che l'inverso è $a' : a * a' = 1 \bmod N$. Ad esempio, l'inverso di 1 in Z'_{15} è 1, di 2 è 8 e così via. Questo è un gruppo moltiplicativo.

Preso G un gruppo, diremo che $|G| = m$ è l'ordine del gruppo e si può dimostrare che $\forall a \in G, a^m = 1$. Questa notazione varrà per entrambi i gruppi con la differenza che in uno faremo la moltiplicazione per se stesso m volte, nell'altro la somma con se stesso per m volte. Inoltre, se prendiamo S un sottogruppo di G , l'ordine di S divide G , cioè, mantiene le stesse proprietà.

Per quanto riguarda i costi, normalmente ignoriamo quelli delle operazioni di base, tuttavia, nei sistemi crittografici dove abbiamo numeri con migliaia di bit, non possiamo farlo. Tra le varie operazioni, abbiamo:

- $a \bmod N$, costa $O(|a||N|)$.
- $a + b$, per due interi a k bit, richiede $O(k)$.
- $a * b$, ha costo $O(k^2)$.
- $a^m \bmod N$ ha costo $O(|m|k^2)$.

Per quanto riguarda il trovare il massimo comune divisore, si fa' uso dell'**algoritmo di Euclide ricorsivo** per cui:

Dati a e b, calcoliamo $\gcd(a,b)$ come segue

$$a = q_1 b + r_1 \quad 0 < r_1 < b$$

$$b = q_2 r_1 + r_2 \quad 0 < r_2 < r_1$$

.....

$$r_{n-1} = q_n r_n$$

r_n e' il gcd cercato

Si può dimostrare che $\gcd(a,b)$ è il più piccolo intero positivo nell'insieme Z_N , per cui, dato che $\gcd(a,b) = 1$, essendo a e b **coprime** in Z_N , possiamo facilmente calcolare l'inverso di a. Per fare ciò, sfruttiamo il fatto che $\lambda a + \mu b : \lambda, \mu \in Z$, da cui:

$$\lambda a + \mu N = 1$$

$$\lambda a = 1 - \mu N$$

Cioè, λa è distante 1 da un multiplo di N e questo vuol dire che $\lambda a \cong 1 \bmod N$, ossia λ è l'inverso di a. $a \in Z_p^*$ è invertibile se $\gcd(a,p) = 1$, cioè, solo se sono coprimi. Dato che vi è questa congruenza, è possibile applicare il **Teorema Cinese del Resto**:

- $m_1, m_2, \dots, m_n$ interi coprimi e positivi,
- $a_1, a_2, \dots, a_n$ interi.
- Il sistema di congruenze appena descritto ammetta un'unica soluz. modulo $M = m_1 \dots m_n$
- Tale soluzione e'

$$x = \sum_{i=1}^n a_i M_i y_i \bmod M$$

- $M_i = M/m_i$ e $y_i = M_i^{-1} \bmod m_i$.

Preso G un **gruppo**, $m = |G|$ è l'ordine di G. Se $g \in G$, l'ordine di g è il minimo n tale che $g^n = 1$ e, se l'ordine di g è proprio m, allora g è detto **generatore** e, se G ammette un generatore, allora questo è **ciclico**.

Z_b

MULTIPLO DI N $\rightarrow \lambda N \bmod N = 0$

SENZA COSÌ DATO Z_N

Preso un valore $a \in G$, questo è detto **quadrato residuo** se esiste $b \in G : b^2 = a$. Detto in parole semplici, di a si può fare la radice quadrata trovando numeri interi. L'insieme dei quadrati residui è un sottogruppo di G . Ci interesseremo particolarmente a guardare $G = Z_p^*$ dove p è primo. Definiamo come **simbolo di Legendre** la funzione tale $J_p : Z \rightarrow \{-1, 0, 1\}$ nel seguente modo:

MANTIENE LE
STESSE PROPRIETÀ
DI G

$$J_p(a) = \begin{cases} 1 & \text{if } a \in QR(Z_p^*) \\ 0 & \text{if } a \bmod p = 0 \\ -1 & \text{otherwise} \end{cases}$$

Per i quadrati residui varranno le seguenti cose:

- $p > 2$ primo, g generatore di Z_p^* , allora $|QR(Z_p^*)| = \frac{p-1}{2}$. Ciò segue dal fatto che Z_p^* è composto da $p-1$ elementi poiché:

$$Z_p^* = \{x : 0 < x < p \cap \gcd(x, p) = 1\} = \{0, \dots, p-1\}$$

Essendo i quadrati tutti della forma $g^{2i} = g^t : t = 1, \dots, p-1$, noi di questi dobbiamo considerare solo i pari, per cui avremo $\frac{p-1}{2}$.

- $p > 2$ primo e $a \in Z_p^*$, $J_p(a) = a^{\frac{p-1}{2} \bmod p}$. Anche questo si può facilmente dimostrare testando la formula nei vari casi.
- $p > 2$ primo, segue che per ogni generatore avremo che $g^{\frac{p-1}{2}} = -1 \bmod p$.

Il problema di Z_p^* è che per utilizzarlo abbiamo bisogno di avere a disposizione numeri di moltissimi bit e ciò non è ovviamente conveniente per ciò che abbiamo bisogno di fare. Possiamo, invece, utilizzare le **curve ellittiche** a patto di utilizzare una matematica estremamente complessa ma ottenendo le proprietà che ci interessano utilizzando molti meno bit.

ALTO Problema $\rightarrow$ Come fare numeri così grandi?

7.2 Primitive Asimmetriche

Proprio in crittografia simmetrica abbiamo utilizzato i cifrari a blocchi per derivare tutto, dobbiamo procurarci strumenti simili per la crittografia asimmetrica. Per fare ciò, come dicevamo, **legheremo le nostre primitive a problemi di natura matematica di cui abbiamo un'operazione facile da calcolare ma di cui è intrattabile riuscire a fare l'inverso**.

Primo tra tutti è il **Logaritmo discreto**. Sia $|G| = m$ un gruppo ciclico e sia g un generatore, il logaritmo discreto $DLog_{G,g} : G \rightarrow Z_m$ è definito come:

$$DLog_{G,g}(a) = x : g^x = a$$

Questa funzione si congettura essere unidirezionale. Il fatto che il problema risulti intrattabile è perché non abbiamo alcun tipo di ordinamento nella sequenza di risultati di a . Possiamo definire il problema del logaritmo discreto come:

G gruppo ciclico di ordine m, g generatore di G.

```

EspG,gdl (A)
  x ←R Zm; X ← gx; x' ←R A(X)
  If (gx'=X) Return 1
  else Return 0
  
```

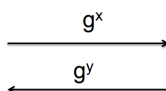
$$\text{Adv}_{G,g}^{\text{dl}}(A) = \Pr[\text{Esp}_{G,g}^{\text{dl}}(A) = 1]$$

bisogna sapere
per l'ultima tipologia
di esercizi

Dove G,g sono noti all'avversario e questo ha come obiettivo quello di indovinare, a partire da X il valore $x : g^x = X$. Diciamo che è un problema DL intrattabile se il vantaggio dell'attaccante è prossimo a 0 per ogni avversario computazionalmente limitato. Ipotizziamo che per alcuni specifici gruppi il logaritmo discreto sia intrattabile, vediamo il **problema Diffie-Hellman computazionale**. Questo problema deriva dalla procedura di scambio delle chiavi definita da Diffie-Hellman che è la seguente:

Alice sceglie $x \leftarrow_R Z_m$
e calcola g^x

Bob sceglie $y \leftarrow_R Z_m$
e calcola g^y



Chiave condivisa: g^{xy}

SCAMBIO DELLE
CHIAVI DH

Da qui possiamo definire il seguente esperimento:

G gruppo ciclico di ordine m, g generatore di G

```

EspG,gcdh (A)
  x,y ←R Zm; X ← gx; Y ← gy;
  Z ←R A(X,Y)
  If (Z=gxy) Return 1
  else Return 0
  
```

$$\text{Adv}_{G,g}^{\text{cdh}}(A) = \Pr[\text{Esp}_{G,g}^{\text{cdh}}(A) = 1]$$

CDH → DIFFIE-HELLMAN
COMPUTAZIONALE

Per cui, il problema è intrattabile se il vantaggio dell'avversario è prossimo a zero per ogni avversario computazionalmente limitato. È chiaro che:

$$\text{Adv}_{G,g}^{\text{dl}}(A) \leq \text{Adv}_{G,g}^{\text{cdh}}(A)$$

E, dunque, il solo modo che abbiamo per risolvere il problema CDH è proprio quello di risolvere il problema DL. Tuttavia, per garantire che la scelta della chiave condivisa sia davvero random, possiamo definire il seguente esperimento:

G gruppo ciclico di ordine m, g generatore di G

$\text{Esp}_{G,g}^{\text{ddh}-1}(A)$	$\text{Esp}_{G,g}^{\text{ddh}-0}(A)$
$x, y \leftarrow_R Z_m; z \leftarrow xy \text{ mod } m$	$x, y, z \leftarrow_R Z_m;$
$X \leftarrow g^x; Y \leftarrow g^y; Z \leftarrow g^z;$	$X \leftarrow g^x; Y \leftarrow g^y; Z \leftarrow g^z;$
$d \leftarrow_R A(X, Y, Z)$	$d \leftarrow_R A(X, Y, Z)$
Return d	Return d

DDH → DECISIONAL
DIFFIE-HELLMAN

$$\text{Adv}_{G,g}^{\text{cdh}}(A) = \Pr[\text{Esp}_{G,g}^{\text{ddh}-1}(A) = 1] - \Pr[\text{Esp}_{G,g}^{\text{ddh}-0}(A) = 1]$$

Che è detto **problema decisionale Diffie-Hellman**. La chiave viene scelta random se il vantaggio dell'avversario è prossimo a zero per ogni avversario computazionalmente limitato. Possiamo dimostrare che:

$$\text{Adv}_{G,g}^{\text{cdh}}(A) \leq \text{Adv}_{G,g}^{\text{ddh}}(A) + \frac{1}{|G|}$$

Cioè che il problema DDH non può essere più difficile di CDH:

← COSTRUIAMO L'ATTACCANTE

$B^{\text{CDH}}(x, y) \rightarrow Z$ $x = g^x \quad y = g^y \quad z = g^{xy}$
 $\text{Adv}^{\text{CDH}}(B)$

 $A^{\text{DDH}}(x, y, z)$
 Run $B^{\text{CDH}}(x, y) \rightarrow \hat{z}$ → LIMITED
 IF $(\hat{z} == z)$ RETURN 1
 ELSE RETURN 0
 $\text{Adv}^{\text{DDH}}(A)$
 $\Pr[\text{Esp}^{\text{DDH}-1}(A) = 1] = \Pr[\text{Esp}^{\text{CDH}}(B) = 1] = \text{Adv}^{\text{CDH}}(B)$
 $\Pr[\text{Esp}^{\text{DDH}-0}(A) = 1] = \frac{1}{|G|}$ ← SPARANDO A CASO LO PRENDE

Nonostante ciò, abbiamo che, in realtà, il problema decisionale Diffie-Hellman non è intrattabile in Z_p^* questo perché, come vedremo tra poco, esiste un attacco basato sul **simbolo di Legendre**. Questo si basa sul fatto che, in Z_p^* , alcune proprietà relative a questo ~~numero~~ **simbolo**, rimangono anche dopo la moltiplicazione. Per cui:

$A(X, Y, Z)$
 If $J_p(X) = 1$ or $J_p(Y) = 1$
 set $s = 1$
 else set $s = -1$
 If $J_p(Z) = s$ return 1
 else return 0

E ciò si può dimostrare nel seguente modo: Se $Z = g^{xy} = g^{x^y}$, ma se per g^x, g^y vale che $J_a(X) = 1, J_a(Y)$, allora la moltiplicazione sarà una cosa del tipo g^{2x^y} che è un quadrato residuo, cioè $J_a(Z) = 1$, altrimenti, se nessuno dei due lo è, allora $J_a(Z) = -1$. Calcoliamo il vantaggio di questo attaccante:

$$\text{Adv}(A)^{\text{ddh}} = \Pr[\text{Esp}^{\text{ddh}-1}(A) = 1] - \Pr[\text{Esp}^{\text{ddh}-0}(A) = 1] = 1 - 1/2 = 1/2$$

La probabilità che nell'esperimento $ddh = 0$ A dica 1 è uguale alla probabilità che fissato 1 (che può essere 0 o 1 o -1), a caso scegliamo un numero che rende l'if binario vero.

Una cosa parecchio importante che vedremo, è che a noi servono specifici numeri primi per cui i fattori della sua fattorizzazione sono grandi abbastanza per cui non possiamo risolverli in maniera efficiente tramite il teorema cinese del resto. Vengono considerati dei buoni numeri primi quelli della forma $p = 2q + 1$ dove q è un numero primo molto grande e che, quindi, non può essere attaccato mediante l'algoritmo Pohlig-Hellman.

7.3 Calcolare Logaritmi Discreti

I migliori modi per calcolare i logaritmi discreti, sono principalmente di 2 tipi:

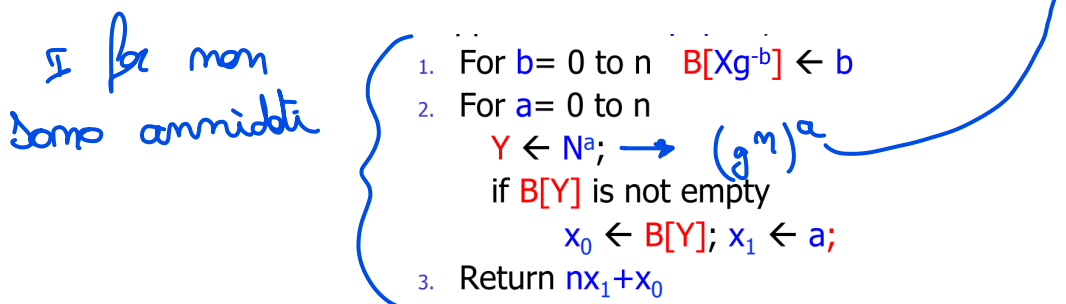
1. **Index Calculus**: sono metodi piuttosto efficienti che però richiedono la presenza di specifiche proprietà aritmetiche per funzionare.
2. **Collision Search**: metodi generici che hanno, però, complessità esponenziale.

Di questi vedremo l'algoritmo **Pohlig-Hellman** che è del primo tipo e l'algoritmo **Baby-step Giant-Step (Shanks)** che è del secondo tipo.

7.3.1 Baby-Step Giant-Step

Questo algoritmo prende in input g un generatore di G e X un elemento in G , e dà in output $x : g^x = X$. Supponiamo che $|G| = m, n = \lceil \sqrt{m} \rceil$. Allora, possiamo porre $x = nx_1 + x_0$ con $0 \leq x_1, x_0 \leq n$, così facendo, noi cerchiamo due numeri anziché uno solo, il che è molto più facile per trovare $X = g^{nx_1+x_0} \rightarrow Xg^{-x_0} = (g^n)^{x_1}$

Per fare ciò l'algoritmo agisce in questo modo:



Il primo ciclo for rappresenta i **baby-step** in cui l'algoritmo calcola prima tutti i lati a sinistra dell'equazione sopra e farà $\sqrt{m}$ passi e, dopodiché farà i **giant step**³ in cui calcolerà tutti i lati a destra dell'equazione e, se trova la posizione occupata, allora abbiamo trovato la collisione e siamo riusciti a rompere il logaritmo discreto. La complessità di questo algoritmo è, quindi $\sqrt{m}$, tuttavia abbiamo la necessità di avere array molto grandi anche se ciò, in realtà, non è un grosso problema dato che lo spazio è riutilizzabile.

³È considerato giant perché qui abbiamo salti più lunghi nell'array.

7.3.2 Pohlig-Hellman

Questo algoritmo funziona solo nel caso in cui la fattorizzazione di $p-1$ è nota. Questo algoritmo, prende in input X , g il generatore in G e la fattorizzazione di $p-1$. Infatti, si possono calcolare DL in tempo:

$$\sum_{i=1}^n a_i (\sqrt{p_i} + |p_i|)$$

Per fare ciò, seguiamo questo ragionamento:

$$p-1 = p_1 \cdot p_2 \cdot p_3$$

Riceviamo $g, h \in \mathbb{Z}_p$ $h = g^{x \bmod p-1} = g^{p_1 \cdot p_2 \cdot p_3}$

$$g^{p_1 \cdot p_2 \cdot p_3} = 1 \bmod p$$

$$h^{p_1 \cdot p_2 \cdot p_3} = 1 \bmod p$$

Primo $h_1 = h^{p_2 \cdot p_3} \quad g_1 = g^{p_2 \cdot p_3}$

Adesso $h_1 = g_1^{x \bmod p_1}$

$$h^{p_1 \cdot p_2 \cdot p_3} = 1 \quad 1 = h^{(p_2 \cdot p_3)^{p_1}} = h_1^{p_1} = 1$$

estrarre $x \bmod p_1$ mi richiede p_1 passi, se uso SHANKS $\rightarrow \sqrt{p_1}$

$h_2 = h^{p_1 \cdot p_3} \quad g_2 = g^{p_1 \cdot p_3}$
Come primo

$h_2 = g_2^{x \bmod p_2}$ (estrarre $x \bmod p_2$ richiede $\sqrt{p_2}$ passi SHANKS)

$h_3 = h^{p_1 \cdot p_2} \quad g_3 = g^{p_1 \cdot p_2}$

$h_3 = g_3^{x \bmod p_3} \rightarrow \sqrt{p_3}$ passi

A questo punto, siamo riusciti a calcolare in tempo $\sqrt{p_1} + \sqrt{p_2} + \sqrt{p_3}$, dunque, abbiamo calcolato $a_i = x \bmod p_i$ con p che sono tutti coprimi tra loro. Dunque, possiamo utilizzare il teorema del resto cinese per calcolare $x \bmod p_1 p_2 p_3$. Da qui, segue quanto detto prima: Se p_1, p_2, p_3 sono piccoli (es. 10 cifre), i calcoli sono immediati. Se invece $p-1 = 2q$ con q enorme, questo trucco non serve a nulla perché fare brute forcing su $\sqrt{q}$ resta impossibile da calcolare.

7.4 Problema della Fattorizzazione

Sia P^k insieme dei primi di taglia k .

$$f : P^k \times P^k \rightarrow N, f(p, q) = pq$$

Cioè, p e q rappresentano la fattorizzazione di N . L'algoritmo più rapido conosciuto per fare ciò è **Number Field Sieve** che ha complessità superpolinomiale. Oggi, fattorizzare numeri di 2048 bit è considerato intrattabile.

7.4.1 RSA

CHIAVE PUBBLICA

Sia N il prodotto di due primi p, q e e un esponente pubblico. La funzione RSA è definita come:

$$RSA[N, e](x) = x^e \pmod{N}$$

Questa è la prima **funzione trapdoor** per cui, dati e, N e $y = x^e \pmod{N}$, è possibile ritrovare x solo se si ha la conoscenza di p, q . Sia $N = pq$, consideriamo il gruppo Z_N^* tale che:

$$Z_N^* = \{x : x < N \cap \gcd(x, N) = 1\} = \{0, \dots, N-1\}$$

Il gruppo ha ordine $\varphi(N) = (p-1)(q-1)$, dunque per il Teorema di Eulero $x^{\varphi(N)} \equiv 1 \pmod{N}$.⁴

Applicando l'algoritmo di Euclide esteso a $(e, \varphi(N))$, sappiamo che esistono interi λ, μ tali che $\lambda e + \mu \varphi(N) = 1$. Per coerenza con la letteratura RSA, poniamo $\lambda = d$.

Questo ci permette di calcolare l'inverso modulare d tale che:

$$ed \equiv 1 \pmod{\varphi(N)}$$

Possiamo ora dimostrare la correttezza dell'algoritmo. Dato il messaggio cifrato $y = x^e \pmod{N}$, per recuperare x calcoliamo:

$$\begin{aligned} y^d &\equiv (x^e)^d \pmod{N} \\ &\equiv x^{ed} \pmod{N} \\ &\equiv x^{1+k\varphi(N)} \pmod{N} && (\text{poiché } ed - 1 \text{ è multiplo di } \varphi(N)) \\ &\equiv x \cdot (x^{\varphi(N)})^k \pmod{N} \\ &\equiv x \cdot (1)^k \pmod{N} && (\text{sostituendo } x^{\varphi(N)} \equiv 1) \\ &\equiv x \pmod{N} \end{aligned}$$

Dunque, se conosco $N = pq$, posso:

1. Calcolare $\Phi(N) = (p-1)(q-1)$
2. Usare l'algoritmo esteso di Euclide su $(e, \Phi(N))$ per trovare d .

⁴ φ indica la funzione totiente di Eulero, legata alla fattorizzazione di N .

3. Utilizzare d per invertire y .

La domanda che ci poniamo adesso è, se io riesco a risolvere il problema RSA, posso risolvere anche il problema della fattorizzazione? Ci sono vari casi da tenere in considerazione:

1. Supponiamo di avere un algoritmo E_1 che dato N calcola $\Phi(N)$, cioè $E_1(N) \rightarrow \varphi(N)$, allora ho risolto il problema della fattorizzazione.
2. Supponiamo di avere un algoritmo $E_2(N, e) \rightarrow d$, allora da questo algoritmo posso ricavare $ed = 1 + \varphi(N)$ e, quindi, $ed - 1 = t\varphi(N)$, e, quindi, calcolare la chiave dato che sappiamo che $ed - 1$ è di sicuro un multiplo di $\varphi(N)$.
3. Se abbiamo, invece, $E_3(N, e, y) \rightarrow x$ abbiamo effettivamente risolto RSA, tuttavia non possiamo concludere nulla riguardo alla fattorizzazione di N . Dunque, i due problemi non sono equivalenti.

A questo punto abbiamo alcuni problemi di natura matematica che dobbiamo tenere in considerazione:

1. Abbiamo abbastanza numeri primi?
2. Supponendo che ve ne siano abbastanza, come faccio a dire che per ogni "dimensione" ci sono abbastanza numeri primi da non permetterci di scegliere contemporaneamente lo stesso?

La risposta alla prima domanda è sì e deriva da una dimostrazione per assurdo effettuata dal matematico Euclide. Alla seconda domanda, invece, risponde il **Prime Number Theorem**:

Sia $\pi(n)$ la **funzione di distribuzione dei primi** (numero di primi minori o uguali a n).

Teorema [Prime Number Theorem]

$$\lim_{n \rightarrow \infty} \frac{\pi(n)}{n / \ln n} = 1$$

L'approssimazione è accurata anche per n piccolo

Ciò vuol dire che se io prendo un intero a caso tra 1 ed n , allora questo sarà primo con probabilità $1/\ln(n)$. Questo ci rassicura molto, tuttavia, non ci dice nulla sul come capire se un numero è primo o no. Un metodo piuttosto conosciuto è quello basato sul **crivello di Eratostane** che, tuttavia, non è per nulla efficiente. Utilizziamo, invece, l'algoritmo **Miller-Rabin**.

7.4.2 Algoritmo Miller-Rabin

L'idea alla base di questo algoritmo è quella di cercare una prova che il numero in input sia composito. Se il test restituisce **Composito**, allora la risposta è corretta con probabilità 1, se restituisce **Primo**, la risposta è sbagliata con probabilità 2^{-2s} . Per come è costruito l'algoritmo, il test ha complessità $O(s|a|^3)$, vediamo l'algoritmo:

Sia $p = 2^k m + 1$ (m dispari)

a (random) t.c. $1 < a \leq p-1$

$b = a^m \bmod p$

If $b = 1 \bmod p$ **then** return "prime"

For $i=0$ **to** $k-1$

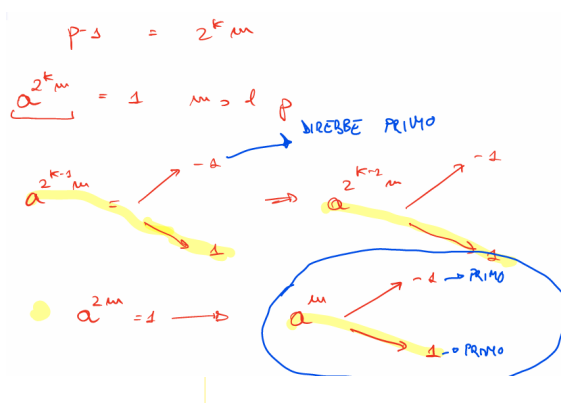
If $b = -1 \bmod p$ **then** return "prime"

else $b = b^2 \bmod p$

Return "composite"

$-1 \bmod p = p-1$ il primo
sugli sicuramente 1
nella sequenza

Dimostriamo che l'algoritmo funziona nel caso in cui p sia primo. Allora, in questo caso, dato che $p - 1 = 2^k m$, avremo che $a^{2^k m} = 1 \bmod p$, per cui facciamo di volta in volta la radice quadrata modulo p che potrebbe ritornare due valori che sono o -1 o 1^5 , nel caso in cui dica -1 , allora è un numero primo, altrimenti si reitera il processo effettuando nuovamente la radice quadrata fino ad arrivare a $a^{2^m} = 1$ e, infine, a a^m che risponderà primo a prescindere dal fatto che il modulo sia 1 o -1 .



A questo punto la domanda che ci rimane è, come generiamo dei primi del tipo $p = 2q + 1$, l'unica cosa che possiamo fare è generare dei numeri primi q , effettuare $2q + 1$ e verificare che p sia primo a sua volta. Per ottimizzare questo processo, genererò solo q dispari ed effettuerò il test di primalità Miller Rabin su $p = 2q + 1$ solo su candidati che hanno già passato alcuni test per verificare che non siano multipli di 3,5,7 ecc.

A questo punto ci chiediamo come effettuare l'elevamento a potenza di RSA. Banalmente si potrebbe pensare di agire in maniera ricorsiva mediante il classico algoritmo:

```
Exp(x, e)
  if e == 0 output 1
  else if e == 1 output x
  else to x * Exp(x, e-1)
```

⁵Stiamo cercando un numero x tale che $x^2 \equiv 1 \pmod{p}$ e questo può valere solo per 1 o $p-1$ nel caso dei numeri primi. Nel caso in cui il numero sia composto, potrebbe valere questa condizione solo per un a particolare, motivo per cui l'algoritmo può sbagliare.

Tuttavia, una ottimizzazione che possiamo fare è la seguente:

```

Exp(n, e)
if e == 0 output x
if e pari output (Exp(n, e/2))^2
if e dispari output x * (Exp(n, e/2))^2
O(log e)

```

Che, come possiamo vedere, ci permette di passare da una complessità $O(e)$ a una $O(\log(e))$. Se volessimo, invece, utilizzare una versione iterativa al posto di quella ricorsiva, esiste il metodo **Square and Multiply** che parte dall'idea di calcolare $x^e \bmod N$ sfruttando la rappresentazione binaria di e .

$$x^e \bmod N = (x^{2^k})^{e_k} \cdot (x^{2^{k-1}})^{e_{k-1}} \cdots (x^2)^{e_1} \cdot x^{e_0} \bmod N$$

1. $d = 1$
2. **for** $i = k$ **downto** 0
3. $d = d \cdot d \bmod N$
4. **if** $e_i = 1$
5. $d = d \cdot x \bmod N$
6. **return** d

Capitolo 8

Cifrari Asimmetrici

Passiamo adesso finalmente ai cifrari asimmetrici. Sostanzialmente, tutti questi cifrari si basano su Diffie-Hellman e RSA. Come sempre, avremo un algoritmo randomizzato di **generazione della coppia di chiavi** (sk, pk) , un algoritmo randomizzato di Encryption che prende in input una chiave pk e un messaggio m e lo cifra in C e, infine, l'algoritmo di Decryption che da (sk, C) produce m .

Una cosa da attenzionare è che messaggi e critttesti sono elementi di gruppi specifici, per cui, il messaggio iniziale sarà trasformato secondo alcune regole al fine di renderlo parte del gruppo a cui l'algoritmo di cifratura fa' riferimento.

Andiamo a dare la solita nozione di sicurezza che sarà in parte simile a quella già vista ma con alcune piccole differenze. Ad esempio, l'avversario non avrà più bisogno di un oracolo per cifrare i suoi messaggi poiché la chiave sarà pubblica e, quindi, l'avversario potrà farlo da solo. Supporremo che, quindi, l'attaccante farà un solo accesso all'oracolo di cifratura che sarà la domanda della sfida. Definiamo, quindi, l'esperimento **ind-cpa**, definito nel seguente modo:

- $\mathcal{AE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ cifrario asimmetrico

$\text{Esp}_{\mathcal{AE}}^{\text{ind-cpa-1}}(A)$	$\text{Esp}_{\mathcal{AE}}^{\text{ind-cpa-0}}(A)$
$(pk, sk) \leftarrow_R \text{KeyGen}$	$(pk, sk) \leftarrow_R \text{KeyGen}$
$b \leftarrow A^{\text{Enc}_{pk}(\text{LR}(\cdot, \cdot, 1))}$	$b \leftarrow A^{\text{Enc}_{pk}(\text{LR}(\cdot, \cdot, 0))}$
Return b	Return b

$$\text{Adv}^{\text{ind-cpa}}(A) = |\Pr[\text{Esp}_{\mathcal{AE}}^{\text{ind-cpa-1}}(A) = 1] - \Pr[\text{Esp}_{\mathcal{AE}}^{\text{ind-cpa-0}}(A) = 1]|$$

Anche in questo caso, il cifrario sarà indistinguibile da attacchi cpa se il vantaggio dell'attaccante è prossimo a zero per ogni avversario computazionalmente limitato. A differenza della sua controparte simmetrica, l'esperimento non impone che i due messaggi abbiano la stessa lunghezza poiché le trasformazioni che questi subiscono per entrare nel gruppo, li renderanno comunque con la stessa lunghezza.

Una nozione ancora più forte sarà quella data dall'esperimento **ind-cca**, cioè, di indistinguibilità da attacchi a critttesto scelto, ossia, l'attaccante ha a disposizione anche un oracolo di decifratura. Questo esperimento è definito a questo modo:

AE=(KeyGen, Enc, Dec) cifrario simmetrico

$\text{Esp}_{\text{SE}}^{\text{ind-cca-1}}(A)$ $(pk, sk) \leftarrow_R \text{KeyGen}$ $b \leftarrow A^{\text{Enc}_{pk}(\text{LR}(\cdot, 1)), \text{Dec}_{sk}(\cdot)}$ If A imbroglia Return 0 else return b	$\text{Esp}_{\text{SE}}^{\text{ind-cca-0}}(A)$ $(pk, sk) \leftarrow_R \text{KeyGen}$ $b \leftarrow A^{\text{Enc}_{pk}(\text{LR}(\cdot, 0)), \text{Dec}_{sk}(\cdot)}$ If A imbroglia Return 0 else return b
--	--

$$\text{Adv}^{\text{ind-cca}}(A) = |\Pr[\text{Esp}_{\text{AE}}^{\text{ind-cca-1}}(A) = 1] - \Pr[\text{Esp}_{\text{AE}}^{\text{ind-cca-0}}(A) = 1]|$$

A imbroglia se interroga $D_k(\cdot)$ su un crittosteso già restituito da $\text{Enc}_{sk}(\text{LR}(\cdot, 1))$

Questo modello è molto più rilevante che nel caso simmetrico e questo è dovuto ad alcune tipologie di attacchi che potremmo adottare:

- **Challenge-Response:** Supponiamo un attaccante chieda in qualche modo alla vittima di dare prova di essere lui. La vittima per fare ciò ha la necessità di tradurre qualcosa con la sua chiave privata. Da qui, segue l'importanza del modello cca poiché spesso si hanno situazioni dove un attaccante può, in un qualche modo, ingannare un server o la stessa vittima per far sì che questo traduca i suoi messaggi.
- **Asta:** Immaginiamo di avere due computer che partecipano a un'asta gestita da un server. Per potervi partecipare, questi dovranno inviare la loro proposta al server in maniera tale che questa sia cifrata con la sua chiave pubblica. Una volta che il server riceve le puntate, questo le decifra e vince la puntata maggiore. Per far sì di avere la vittoria assicurata, un computer potrebbe barare prendendo il messaggio dell'avversario e aggiungendogli tramite alcune trasformate, 1. Così facendo non solo vincerebbe sicuramente l'asta ma riuscirebbe a farlo con il minor dispendio possibile. Questa cosa non è fattibile nel caso in cui un modello sia ind-cca, poiché non si avrebbe modo di conoscere l'effetto delle trasformazioni che effettuiamo.

P_k del
BAUDRORE

Il concetto descritto nel caso dell'asta è di particolare interesse ed è una proprietà nota come **non malleabilità**. Diremo che un cifrario sicuro in senso ind-cca garantisce non malleabilità. Sia $AE = (\text{KeyGen}, \text{Enc}, \text{Dec})$ un cifrario asimmetrico sicuro in senso ind-cpa. Sia $F : M \rightarrow M$ una funzione efficacemente computabile. Sia AE malleabile relativamente ad F, allora esiste una procedura $\text{MAUL} : \text{MAUL}(\text{Enc}(m), F) \rightarrow \text{Enc}(F(M))$. Se un cifrario permette ciò, allora non può essere sicuro in senso ind-cca. Possiamo verificare questa cosa generando il seguente attaccante:

A(AE, F)
 $m_0, m_1 \leftarrow_R \mathcal{M};$
 $C \leftarrow O_E(m_0, m_1);$
 $C' \leftarrow \text{MAUL}(C, F)$
 $m \leftarrow O_D(C')$
 IF ($m = F(m_1)$) return 1
 ELSE return 0

Andiamo ad analizzare il vantaggio dell'attaccante:

$$\begin{aligned} Pr[Esp^{ind-cca-1}(A) = 1] &= 1 && \text{L'attaccante non può mai sbagliare} \\ Pr[Esp^{ind-cca-0}(A) = 1] &= 0 && \text{Dunque il vantaggio di A sarà :} \\ Adv(A) &= 1 \end{aligned}$$

Nonostante la malleabilità sia un qualcosa che può spaventarci, questa può essere utile in specifici contesti. Un esempio è quello per cui vogliamo effettuare statistica su dati cifrati senza però decifrarli. Contesti del genere possono accadere spesso quando si trattano dati sanitari o magari bancari dove si vogliono rendere anonimi alcuni dati ma si vuole comunque poter effettuare operazioni su di essi. Questa tipologia di cifratura è conosciuta come **Omomorfica** che è un tipo di crittografia che permette di eseguire calcoli su dati cifrati, ottenendo un risultato cifrato che corrisponde a quello che si otterrebbe se i dati fossero in chiaro.

8.1 Hybrid Encryption

Per quanto possiamo impegnarci a creare cifrature asimmetriche sicure, a causa puramente di aspetti che non possiamo modificare, questa sarà sempre più lenta della cifratura simmetrica. Ration per cui nascono i **cifrari ibridi**. Questi sfruttano cifratura simmetrica per la cifratura dei messaggi e cifratura asimmetrica per lo scambio della chiave, di fatto, ottenendo una maggiore velocità e non avendo alcun tipo di problema con la codifica.

Siano $AE = (KeyGen^a, Enc^a, Dec^a)$, $SE = (KeyGen^s, Enc^s, Dec^s)$ rispettivamente un cifrario asimmetrico e uno simmetrico.

Il cifrario $HE = (KeyGen^H, Enc^H, Dec^H)$ è definito dai seguenti algoritmi:

- **KeyGen^H**: Banalmente la generazione della chiave sarà effettuata seguendo l'algoritmo $KeyGen^a$.
- **Enc^H**: Qui arriviamo ad alcuni dettagli più interessanti. Questo algoritmo prenderà come input la chiave privata pk e il messaggio m da inviare. Dopo-diché verrà generata una chiave simmetrica k con cui verrà cifrato il messaggio e, infine, verrà cifrata la chiave simmetrica mediante cifratura asimmetrica con la chiave privata del destinatario. Così facendo verrà inviato $C^s || C^a$ dove C^s contiene la il messaggio e C^a contiene la chiave simmetrica con cui è stato cifrato il messaggio.

```

EncH (pk,m)
k  $\leftarrow_R$  KeyGens ;
Cs  $\leftarrow$  Encs (k,m);
Ca  $\leftarrow$  Enca (pk,k);
return (Cs, Ca)

```

- **Dec^H**: Quando effettueremo la decifratura del messaggio, avremo a nostra disposizione sk la chiave segreta del destinatario e C^s, C^a . Quello che faremo sarà, dunque decifrare C^a per ottenere la chiave simmetrica con cui è stato cifrato il messaggio e, infine, decifrare il messaggio appena ottenuto.

```

DecH (sk, (Cs, Ca))
k ← Deca (sk, Ca);
m ← Decs (k, Cs);
return m

```

La chiave simmetrica scambiata tra i due, sarà definita **chiave di sessione** e questa cambierà, per l'appunto, a ogni nuova sessione. Così facendo, perdendo una chiave di questo tipo, non mettiamo a rischio future sessioni.

8.2 Cifrario El-Gamal

Il cifrario El-Gamal fu uno dei primi cifrari asimmetrici, questo è definito su un gruppo G di ordine q e sia g un generatore di questo gruppo. Possiamo definire El-Gamal in questo modo:

- **KeyGen**: L'algoritmo di generazione della chiave sceglie un x random tra 1 e q e si prende $h = g^x$. La chiave pubblica $pk = (g, h, q)$ mentre la chiave privata è $sk = (x)$.
- **Enc(PK, M)**: L'algoritmo di Encryption genera una randomness r tra 0 e q . Dopodiché pone $C_1 = g^r$ e $C_2 = h^r M$. In questo caso, come possiamo notare non abbiamo alcun tipo di trasformazione del messaggio poiché lo spazio dei messaggi corrisponde proprio al gruppo.
- **Dec(SK, C₁, C₂)**: Questo algoritmo prende $A = C_1^x$ e da questo avremo che $M = \frac{C_2}{A}$. Per capire questa cosa, vediamo cosa accade.

$$A = C_1^x = (g^r)^x = g^{rx} = h^r$$

$$C_2 / A = \frac{h^r \cdot M}{h^r} = M$$

Il cifrario El-Gamal garantisce sicurezza in senso ind-cpa se il problema DDH è intrattabile in G .

Supponiamo che B sia un avversario contro la sicurezza ind-cpa per El-Gamal. Cerchiamo di costruire A un avversario che sfrutta B per rompere DDH.

$A(x, y, z):$
 $g, h = y, q \leftarrow \text{Sceglie una TRIPETTA}$
 $PK = (h, g, q)$
 $SK = \emptyset$
 $(M_0, M_1) = B(PK) \leftarrow \text{pretti } M_0, M_1 \text{ random}$
 $b \leftarrow_R \{0, 1\}$
 $C_1 \leftarrow X; C_2 \leftarrow ZH_b$
 $b' \leftarrow B(C_1, C_2)$
 IF $(b' = b)$ RETURN 1
 ELSE RETURN 0

Cerchiamo di capirlo meglio guardando il vantaggio di A:

$$\begin{aligned}
 \Pr[\text{Exp}^{\text{ddh}-1}(A) = 1] &= \Pr[b' = b] = \\
 &= \Pr[b' = b \mid b = 1] \cdot \Pr[b = 1] + \Pr[b' = b \mid b = 0] \cdot \Pr[b = 0] = \\
 &= \frac{1}{2} \Pr[b' = 1 \mid b = 1] + \frac{1}{2} \Pr[b' = 0 \mid b = 0] \\
 &= \frac{1}{2} + \frac{1}{2} \left[\Pr[B = 1 \mid b = 1] - \Pr[B = 1 \mid b = 0] \right] \rightarrow \Pr[\text{Exp}^{\text{ddh}-1}(A) = 1] = \frac{1}{2} + \frac{1}{2} \text{Adv}(B) \\
 &\quad \underbrace{\hspace{10em}}_{\text{Adv}(B)^{\text{ind-CPA}}} \quad \Pr[\text{Exp}^{\text{ddh}-0}(A) = 1] = \frac{1}{2} + \frac{1}{2} \underset{0}{\text{Adv}(B)}
 \end{aligned}$$

Se siamo in $\text{ddh}-1$

$$x = g^x \quad y = g^y \quad z = g^{xy}$$

$$C_1 = g^x \quad C_2 = z m_b = g^{xy} m_b = h^y m_b \rightarrow \text{Tutto sta alla brancia di } B \text{ perché è una cifratura di EL-Grass}$$

Se siamo in $\text{ddh}-0$

$$x = g^x \quad y = g^y \quad z = g^z$$

$$C_1 = g^x \quad C_2 = g^z m_b \rightarrow \text{ma } z \text{ è casuale, dunque } C_2 \text{ non può essere decifrato}$$

Essendo anche x a caso, B è come se dovesse pure a caso

Per cui, avremo che il vantaggio $\text{Adv}(A) = 1/2 \text{Adv}(B)$.

Un'altra considerazione da fare è che questo cifrario è **malleabile**, infatti, possiamo vedere:

$$\begin{aligned}
 C_1 &= g^x & C_2 &= h^z M_1 \\
 \hat{C}_1 &= g^{\hat{x}} & \hat{C}_2 &= h^{\hat{z}} M_2 \\
 & & & g^{z+\delta} \quad h^{z+\delta} M_1 M_2
 \end{aligned}$$

Tuttavia, possiamo effettuare solo operazioni di moltiplicazione in questo caso. Per rendere possibili le addizioni, abbiamo bisogno di fare alcune modifiche e scendere a compromessi.

8.2.1 Lifted El-Gamal

Sia B piccolo abbastanza da poter effettuare brute-force. Il cifrario Lifted El-Gamal sarà quindi definito così:

$$\begin{array}{l}
 \text{KEY GEN: } G, g \quad x \leftarrow \{1 \dots q\} \quad h = g^x \\
 P_K(h, g, q, B) \quad S_K = x \\
 m = \{0, \dots, B\} \\
 \hline
 \text{Enc } (P_K, m): \\
 r \leftarrow \{1, \dots, q\} \\
 C_1 = g^r \quad C_2 = h^r \cdot g^m \\
 \hline
 \text{Dec } (S_K, C_1, C_2) \\
 A \leftarrow C_1^x; \quad D = C_2 / A = \frac{h^r g^m}{h^r} = g^m \\
 \text{For } i = 0 \text{ to } B: \\
 \quad \text{If } g^i = D \quad \text{RETURN } i \\
 \text{RETURN } \perp
 \end{array}$$

Vediamo, infatti:

$$\begin{array}{l}
 C_1 = g^r \quad C_2 = h^r g^{m_1} \\
 \hat{C}_1 = g^s \quad \hat{C}_2 = h^s g^{m_2} \\
 g^{r+s} \quad h^{r+s} g^{m_1+m_2}
 \end{array}$$

8.3 Cifrario Paillier

Passiamo adesso a un cifrario omomorfico che non ha tutte le problematiche che ha Lifted El-Gamal. Questo approccio sarà un ibrido tra una soluzione basata sul logaritmo discreto e una basata sulla fattorizzazione poiché cerca di prendere alcune caratteristiche da ognuno:

- **Logaritmo discreto:** Tramite le soluzioni basate sul logaritmo discreto, riusciamo a ottenere omomorfismo. Tuttavia, non siamo in grado di estrarre g^{x+y} .
- **Fattorizzazione:** Siamo in grado di costruire un sistema trapdoor come RSA. Tuttavia, RSA è moltiplicativo, per cui:

$$x^e \bmod N * y^e \bmod N = (xy)^e \bmod N$$

8.3.1 Un po' di Algebra (Ancora...)

Ricordiamoci che:

$$Z_N^* = \{x : 0 < x < N \cap \gcd(x, N) = 1\}$$

Dove $N = pq$ con p, q numeri primi. Consideriamo adesso $Z_{N^2}^*$ con $N^2 = p^2q^2$. Definiamo adesso x un **N-Residuo** se $\exists y \rightarrow y^N \equiv x \pmod{N^2}$ dove y è radice perfetta N-Esima. Supponiamo di prendere l'insieme T sottogruppo di $Z_{N^2}^*$. Nello specifico il sottogruppo generato dall'elemento $(1+N)$:

$$T = \{(1 + xN) \in Z_{N^2}^* : x \in Z_N\}$$

Vediamo che tipo di elementi abbiamo:

$$\begin{aligned} x = 0 &\rightarrow 1 \\ x = N &\rightarrow 1 + N^2 \pmod{N^2} = 1 \end{aligned}$$

Andando sul generico, otteniamo:

$$\begin{aligned} (1 + xN)^N \pmod{N^2} &= \\ &= \sum_{i=0}^N \binom{N}{i} (xN)^i \pmod{N^2} = \\ &= 1 + \binom{N}{1} xN + \sum_{i=2}^N \binom{N}{i} (xN)^i \pmod{N^2} = \\ &= 1 + xN^2 + \sum_{i=2}^N \binom{N}{i} (xN)^i = 1 \end{aligned}$$

$$\text{Vediamo che } xN^2 = 0 \text{ e } \sum_{i=0}^N \binom{N}{i} (xN)^i = 0 \text{ in quanto tutti multipli di } N^2$$

Quindi, possiamo concludere che tutti gli elementi in T sono uguali a 1 in $Z_{N^2}^*$. Da qui derivano alcune considerazioni:

- Ogni N-residuo ammette N radici N-esime distinte. Cioè se ho x^N questo può derivare da N elementi diversi.
- Si consideri l'insieme $T = \{(1 + xN) \in Z_{N^2}^* : x \in Z_N\}$, ogni elemento in T è tale che $z^N = 1 \pmod{N^2}$.
- L'ordine di $Z_{N^2}^*$ è $\phi(N^2)$, dunque per ogni x in $Z_{N^2}^*$ si ha che $x^{\phi(N^2)} \pmod{N^2} = 1$.

Sappiamo $\phi(N^2) = N\phi(N)$, dunque $x^{\phi(N^2)} = 1 \pmod{N^2}$ è equivalente a scrivere $x^{N\phi(N)} = 1 \pmod{N^2}$. Supponendo di prendere un elemento w che può essere con probabilità $1/2$ un N-Residuo e con probabilità $1/2$ un elemento casuale in $Z_{N^2}^*$. Conoscendo la fattorizzazione di N , possiamo capire se questo è un N-residuo seguendo questo algoritmo:

$A(\phi(w), w)$
 if $w \phi(w) = 1 \bmod N^2$
 output 1
 else output 0

Se la fattorizzazione è ignota non esiste nessun algoritmo polinomiale per risolvere questo problema. Possiamo definire il problema della N residuosità decisionale in questo modo:

$\text{Esp}_N^{\text{DCRA-1}}(A)$ $x \leftarrow_R Z_{N^2}^*$ $w \leftarrow x^N \bmod N^2$ $d \leftarrow_R A(w)$ Return d	$\text{Esp}_N^{\text{DCRA-0}}(A)$ $w \leftarrow_R Z_{N^2}^*$ $d \leftarrow_R A(w)$ Return d
--	--

$$\text{Adv}_{N,A}^{\text{DCRA}} = \Pr[\text{Esp}_N^{\text{DCRA-1}}(A) = 1] - \Pr[\text{Esp}_N^{\text{DCRA-0}}(A) = 1]$$

La probabilità che in DCRA-0 si prenda un w che è un elemento residuo è $1/N$, che è praticamente nulla. Si può dimostrare nel seguente modo: l'insieme $Z_{N^2}^*$ ha $\phi(N^2) = N\phi(N)$ e la cardinalità degli N -residui è $\phi(N)$. Quindi, casi favorevoli su casi totali arriviamo a $1/N$.

Un'ulteriore considerazione è che ogni elemento $w \in Z_{N^2}^*$ può essere scritto nella forma $(1 + xN)y^N$ con $x \in Z_N$ e $y \in Z_N^*$. Sia $Z_{N^2}^* \cong Z_N \times Z_N^*$ un isomorfismo, fissato $c = (1 + N)y^N \bmod N^2$, esiste una sola coppia (x, y) che dà quel c .

8.3.2 Algoritmo di Cifratura

Innanzitutto definiamo l'algoritmo di generazione della chiave. Questo è molto simile a RSA, generiamo p, q primi a caso e calcoliamo $N = pq$. Scegliamo come chiave pubblica N e come chiave privata la coppia (p, q) , cioè la fattorizzazione di N . Lo spazio dei messaggi è Z_N e lo spazio dei crittoteesti è $Z_{N^2}^*$.

L'algoritmo di cifratura è, invece, il seguente:

Enc(N, m) // $m \in Z_N$
 $y \leftarrow_R Z_N^*$;
 $c = (1 + mN)y^N \bmod N^2$
 Return c

Più complicata è la decifratura dei messaggi. Cerchiamo di dividere la cosa in più passi:

1. Calcoliamo:

$$\begin{aligned}
 c^{\phi(N)} &= \\
 &= ((1 + mN)y^N)^{\phi(N)} \mod N^2 = \\
 &= (1 + mN)^{\phi(N)} y^{N\phi(N)} \mod N^2 = \\
 &= (1 + mN)^{\phi(N)} \quad \text{PER LE PROPRIETÀ VISTE PRIMA È 1}
 \end{aligned}$$

In questo modo, togliendo y , abbiamo tolto la randomness.

2. Ogni elemento $(1 + xN)$ ha ordine N in $Z_{N^2}^*$. Dunque, poiché $\gcd(N, \phi(N)) = 1$, esiste d tale che $d\phi(N) = 1 \mod N$.

$$C = (c^{\phi(N)})^d = (1 + mN)^{\phi(N)d} \mod N^2 = (1 + mN) \mod N^2$$

Il fatto che $d\phi(N) = 1 \mod N$, vuol dire che $d\phi(N) = 1 + kN$. Posso scomporre $(1 + mN)^{1+kN} = (1 + mN) * ((1 + mN)^N)^k = (1 + mN)$. MULTIPLO DI N CHE SI SEMPLIFICA CON IL MODULO

3. Infine, possiamo calcolare m da c nel seguente modo:

$$m = \frac{C - 1}{N}$$

Vediamo come questo cifrario è malleabile e come si possa fare la somma:

$$\begin{aligned}
 C_1 &= E_{m_1}(m_1) = (1 + m_1N) y_1^N \mod N^2 \\
 C_2 &= E_{m_2}(m_2) = (1 + m_2N) y_2^N \mod N^2 \\
 C_1 \cdot C_2 \mod N^2 &= \\
 (1 + m_1N) y_1^N \cdot (1 + m_2N) y_2^N \mod N^2 &= \\
 (1 + m_1N) \cdot (1 + m_2N) y_1^N y_2^N \mod N^2 &= \\
 (1 + m_1N + m_2N + \underbrace{m_1m_2N^2}_0) y_1^N y_2^N \mod N^2 &= \\
 (1 + (m_1 + m_2)N) y_1^N y_2^N \mod N^2
 \end{aligned}$$

$m \in \mathbb{Z}_N \quad y \in \mathbb{Z}_N^*$

Notiamo come la prima parte abbia effettivamente la somma:

ANALIZZIAMO LA SECONDA PARTE

$$y_1^N \cdot y_2^N \bmod N^2 = (y_1 \cdot y_2)^N \bmod N^2 = y^N \bmod N^2$$

Se $y_1 \in \mathbb{Z}_N^*$ e $y_2 \in \mathbb{Z}_N^*$ non è detto che $y_1 \cdot y_2 \bmod N^2$ sia in $\mathbb{Z}_N^*$

Se mettiamo $(y_1 \cdot y_2)^N \bmod N^2$ può essere $\mathbb{Z}_N^*$

$$y_1 \cdot y_2 \bmod N^2 = a + bN$$

$$(a + bN)^N \bmod N^2 = \sum_{i=0}^N \binom{N}{i} a^{N-i} (bN)^i \bmod N^2 =$$

$$= a^N + \underbrace{N \cdot a^{N-1} \cdot bN}_{=0} + \underbrace{\sum_{i=2}^N \binom{N}{i} a^{N-i} (bN)^i}_{i \text{ è almeno due } = 0} \bmod N^2 =$$

$$= a^N \bmod N \quad \leftarrow \text{difficilmente faccio somme dopo il mod } N$$

se N è di 2000 bit

Adesso che siamo riusciti a fare la somma, vediamo come fare la **moltiplicazione per una costante**:

$$c \quad \alpha \in \mathbb{N} \quad \alpha \cdot m \bmod N$$

$$c = (1 + mN) y^N \bmod N^2$$

$$c^\alpha \bmod N^2 = (1 + mN)^\alpha \cdot \underbrace{(y^N)^\alpha}_{y^N} \bmod N^2 =$$

$$(1 + mN)^\alpha \bmod N^2 = \sum_{i=0}^{\alpha} \binom{\alpha}{i} (mN)^i \bmod N^2$$

$$= 1 + \alpha mN + \underbrace{\sum_{i=2}^{\alpha} \binom{\alpha}{i} (mN)^i}_{=0} \bmod N^2$$

$$= (1 + \alpha mN)$$

8.3.3 Elezione Multi-Candidato

Supponiamo di avere quattro candidati in un sistema di elezione che chiameremo $C_1, \dots, C_4$ e vogliamo che vi sia segretezza nel voto ma che sommando tutti i voti

ci venga fornito il risultato delle elezioni. Sappiamo che lo spazio dei messaggi è Z_N con $N = pq$, immaginando che $N = 2000$, possiamo creare un array di questo tipo:

500	500	500	500
0	0	0...1	0

In questo modo, dividiamo l'array in quattro sezioni e ogni voto si va' a sommare in quella specifica sezione. Questo sistema, così come proposto, non è funzionante poiché presuppone che tutti gli elettori siano onesti. Tuttavia, vi sono dei modi per far sì che questo sistema funzioni.

8.4 Cifrari RSA-OAEP

Fino a ora abbiamo presentato tutti cifrari sicuri contro avversari CPA ma che crolla in contesti CCA. Passiamo adesso a un cifrario sicuro contro attacchi CCA noto come **RSA-OAEP** che è l'attuale standard di sicurezza anche se, in questi ultimissimi anni, si sta passando ai cifrari post-quantum. Questo cifrario è sicuro se considerato nel contesto **Random Oracle Model** in cui abbiamo la necessità di un oracolo di Hash che preso in input x ritorna il suo hash in maniera tale che sia del tutto casuale. La nostra ipotesi è che se sostituiamo a questo oracolo una vera funzione Hash, il cifrario rimanga sicuro. Questa ipotesi è, però, falsa; tuttavia, tutti i controesempi trovati sono molto artificiali e del tutto impraticabili. La difficoltà nel costruire cifrari sicuri contro attacchi a crittotesto scelto è dovuta principalmente a due cause:

1. Il simulatore deve essere in grado di decifrare senza risolvere il problema computazionale alla base della sicurezza del sistema.
2. Dobbiamo impedire all'attaccante di creare crittotesti utili a partire da quello sul quale lo sfidiamo.

Prima di iniziare con il cifrario vero e proprio, diamo alcune definizioni:

- k è la "taglia" del modulo RSA in uso.
- k_0, k_1 sono dei parametri tali che $k_0 + k_1 < k$.
- Lo spazio dei messaggi è composto da tutte le stringhe di lunghezza n dove $n = k - k_0 - k_1$.
- Siano

$$\begin{aligned} - & G : \{0, 1\}^{k_0} \rightarrow \{0, 1\}^{n+k_1} \\ - & H : \{0, 1\}^{n+k_1} \rightarrow \{0, 1\}^{k_0} \end{aligned}$$

Due random oracle.

Cominciamo a vedere gli algoritmi di generazione della chiave, di cifratura e decifratura. L'algoritmo di generazione della chiave è, banalmente, lo stesso algoritmo utilizzato per RSA.

Per quanto riguarda l'algoritmo di cifratura, avremo invece:

```

Enc( $N, m$ )    //  $m \in \{0, 1\}^n$  ADDING
   $r \leftarrow_R \{0, 1\}^{k_0}$ ;
   $s \leftarrow G(r) \oplus (m || 0^{k_1})$ ;    //  $s \in \{0, 1\}^{n+k_1}$ 
   $t \leftarrow H(s) \oplus r$ ;    //  $t \in \{0, 1\}^{k_0}$ ;
   $w \leftarrow s || t$ ;    //  $w \in \{0, 1\}^{n+k_1+k_0}$ 
   $y \leftarrow \text{RSA}(w)$ ;    //  $y \in \{0, 1\}^k$ ;
  Return  $y$ 

```

Handwritten notes:
 - A blue arrow points from m to $(m || 0^{k_1})$ with the word "ADDING".
 - A blue arrow points from w to $\text{RSA}(w)$ with the text "we need".
 - Below the algorithm, $y \rightsquigarrow y'$ is written in blue.

Il costo di questa operazione è molto basso, praticamente la cosa più costosa è l'elevazione causata dall'utilizzo di RSA. Il vero problema è che in termini di banda, non possiamo fare molto di meglio poiché, supponendo di voler mandare solo 256 bit, ne manderemo 3000. Tuttavia diciamo che ci accontentiamo poiché, come già detto precedentemente, quello che facciamo è utilizzare la crittografia asimmetrica solo per scambiare una chiave simmetrica. Dunque, una volta scambiata la chiave simmetrica, la cifratura asimmetrica non viene più utilizzata.

Infine, per quanto riguarda l'algoritmo di decifratura abbiamo:

```

Dec( $N, y$ )    //  $c \in \{0, 1\}^k$ 
   $w \leftarrow \text{RSA}^{-1}(y)$ ;
  Sia  $w = s || t$     //  $s \in \{0, 1\}^{n+k_1}$   $t \in \{0, 1\}^{k_0}$ ;
   $r' \leftarrow H(s) \oplus t$ ;
   $z \leftarrow G(r') \oplus s$ ;
  Sia  $z = m || c$ ;    //  $m \in \{0, 1\}^n$   $c \in \{0, 1\}^{k_1}$ ;
  If  $c == 0^{k_1}$  output  $m$ ;
  else output  $\perp$ .

```

Handwritten notes:
 - A blue arrow points from y to w with the text $y' \rightsquigarrow w'$.
 - Below $w = s || t$, $w' = s' || t'$ is written in blue.

Come possiamo notare, riusciamo effettivamente a decifrare il messaggio e il padding precedentemente effettuato. Proprio su quest'ultimo viene effettuato un ulteriore controllo, poiché con una probabilità molto alta, se viene effettuata una modifica al crittotesto, questo andrà a modificare anche quella stringa di zeri trasformandone almeno uno.

Una volta visto che questo sistema è sicuro (non lo dimostriamo poiché troppo complicato), non ci rimane altro che gestire correttamente il sistema delle chiavi

pubbliche. Tendenzialmente, viene adottato il sistema **Public Key Infrastructure**, una struttura di chiavi che certificano che una data chiave pubblica appartiene a una specifica persona/identità. Questa struttura è parecchio complessa e si arriva ricorsivamente a dover verificare l'autenticità dell'autenticità ecc. fino ad arrivare alla così dette **root key** che sarebbe una chiave auto certificata. Per ottenere la certificazione è necessaria una procedura burocratica atta a questo scopo.

8.5 Identity Based Encryption

Per ovviare al problema della certificazione della chiave, quello che vorremmo fare è sfruttare procedure burocratiche già esistenti per ottenere una chiave pubblica. Ad esempio, ipotizziamo che la procedura d'iscrizione all'università rilasci una mail universitaria e supponiamo di voler utilizzare quest'ultima come chiave pubblica. Questo sistema viene definito **Cifratura basata sull'identità**. Rendiamoci conto dei vantaggi che questo sistema può offrire:

- La mail è facilmente riconoscibile, non c'è il rischio di confondere una "chiave" da un'altra.
- Posso creare delle chiavi pubbliche a "scadenza" ad esempio generando una mail seguita dall'anno di validità.
- Posso inviare messaggi senza che il destinatario abbia già generato la sua chiave.

Tuttavia, rimane il problema della generazione della chiave privata. Per fare ciò, abbiamo bisogno di un **segreto master** da cui poter generare le chiavi private a partire dalle chiavi pubbliche. In questo modo, riusciremo a risolvere anche il problema di **key escrow**, cioè il fatto che, in certi casi particolari (come ad esempio i casi di procedura penale), voglio far sì che un ente terzo possa rompere la cifratura. Questo viene risolto poiché basterebbe che l'ente di generazione generi nuovamente la chiave privata dell'utente e la dia alla corte penale. Rimane però il fatto che, dando questa possibilità, non è detto che non ne venga abusato. Inoltre, possiamo immaginare quanto possa essere dannoso uno scenario in cui venga compromessa la chiave master.

8.5.1 Schema Boneh-Franklin

Questo sistema di cifratura sfrutta mappe **bilineari** tra due gruppi G_1, G_2 in cui il problema Diffie-Hellman è intrattabile. Avremo, quindi, un **Trusted-Third-Party** che genera le chiavi private degli utenti. Vediamo i requisiti di sicurezza che vogliamo ottenere oltre quelli classici:

1. Vogliamo far sì che se non si conosce la master key, non si possa generare la chiave privata.
2. Vogliamo far sì che due utenti o più collusi non possano generare un'altra chiave che non gli appartiene.

Vediamo quindi la definizione **ind-id-cpa**:

CIFRATURA
2 ORACOLI
ESTRAZIONE CHIAVE

IBE=(Setup, KeyDer, Enc, Dec)	
<i>L'ATTACANTE SCEGLIE L'ID* SU CUI ESSERE SFIDATO</i> Esp^{ind-id-cpa-1} (A) $(pk, msk) \leftarrow_R \text{Setup}$ $b \leftarrow A^{\text{Enc}_{pk}(\text{id}^*, \text{LR}(\cdot, \cdot, 1)), \text{KeyDer}_{msk}(\cdot)}$ If A imbrogliava Return 0 else return b	Esp^{ind-id-cpa-0} (A) $(pk, msk) \leftarrow_R \text{Setup}$ $b \leftarrow A^{\text{Enc}_{pk}(\text{id}^*, \text{LR}(\cdot, \cdot, 0)), \text{KeyDer}_{msk}(\cdot)}$ If A imbrogliava Return 0 else return b
$\text{Adv}^{\text{ind-id-cpa}}(A) = \Pr[\text{Esp}^{\text{ind-id-cpa-1}}(A) = 1] - \Pr[\text{Esp}^{\text{ind-id-cpa-0}}(A) = 1]$ A imbrogliava se interroga $\text{KeyDer}_{msk}(\cdot)$ sull'identità id^*.	

In cui A può accedere a un oracolo di estrazione che prende in input un id e ne restituisce la chiave privata e a un oracolo di cifratura che prende una stringa e la cifra con l'id su cui A vuole essere sfidato.

Siano G_1, G_2, G_t dei gruppi con le seguenti proprietà:

1. G_1, G_2 sono gruppi di ordine q primo e G_1 è generato da P e G_2 da P' .
2. Esiste un isomorfismo $\rho : G_2 \rightarrow G_1, \rho(P') = P$
3. Esiste una mappa bilineare $e : G_1 \times G_2 \rightarrow G_t$

Una mappa si dice **Bilineare** se $\forall U \in G_1, V \in G_2, (a, b) \in \mathbb{Z} : e(U^a, V^b) = e(U, V)^{ab}$. La mappa non deve nemmeno essere **degenere**, cioè non deve succedere che $e(P, P') = 1_{G_t}$. Un esempio di mappa bilineare è il prodotto scalare.

Ma vediamo adesso il cifrario in sé per sé. Questo si compone di quattro algoritmi: **Setup**, **Key Extraction**, **Encryption** e **Decryption**. Vediamoli uno a uno:

- **Setup**: Questo sceglie il master secret key uniformemente a caso in $[1, \dots, q]$. La chiave pubblica sarà $Pk = (P')^{msk}$.
- **KeyExtraction**: la chiave segreta dell'utente id sarà $P_{id} = H_1(id)$ e poi $usk = P_{id}^{msk}$. Con H_1 che manda stringhe di bit di lunghezza arbitraria in G_1 .
- **Cifratura**: L'algoritmo funziona nel seguente modo:

Enc(m, id, Pk)
 $r \leftarrow_R \{1, \dots, q\}; P_{id} \leftarrow H_1(id)$
 $k \leftarrow e(P_{id}, Pk)^r;$
 $c_1 \leftarrow (P')^r; c_2 \leftarrow m \oplus H_2(k)$
 Return (c_1, c_2)

Con H_2 che manda G_t in stringhe di bit di lunghezza arbitraria.

USO SECRET KEY

- **Decifratura:** infine, vediamo l'algoritmo di decifratura:

Dec $((c_1, c_2), \text{id}, \text{usk})$
 $k \leftarrow e(\text{usk}, c_1); m \leftarrow c_2 \oplus H_2(k)$
 Return m

Possiamo dimostrare la correttezza di questo algoritmo:

$$\begin{aligned} K &= e(P_{\text{id}}, P_K)^z \rightarrow e(H_2(\text{id}), P^{m_{\text{id}}})^z \rightarrow e(H_2(\text{id}), P)^{m_{\text{id}}z} \\ K &= e(\text{usk}, c_1) \rightarrow e(P_{\text{id}}^{m_{\text{id}}}, P^{z'}) = e(H_2(\text{id})^{m_{\text{id}}}, P^{z'}) = e(H_2(\text{id}), P)^{z m_{\text{id}}} \end{aligned}$$

Per quanto riguarda, invece, la dimostrazione di sicurezza di questo cifrario, utilizziamo il **problema DDH bilineare**, per cui:

$$P^a, P^b, P^c \quad T \leftarrow \begin{cases} e(P, P)^{abc} \\ \text{random in } G_T \end{cases}$$

Dove l'obiettivo è quello di capire se T è $e(P, P)^{abc}$ o un elemento a caso in G_T . Si può dimostrare che il problema BDH può essere risolto se viene risolto il problema CDH.¹

B AVVERSAIO PER CDH

Vogliamo A che usa B per risolvere BDH

$$A(g^a, g^b, g^c) \rightarrow e(g, g)^{abc}$$

$$\text{run } B(g^a, g^b) \rightarrow g^z = g^{ab}$$

$$T = e(g^z, g^c) \rightarrow e(g^{ab}, g^c) \rightarrow e(g, g)^{abc}$$

output

B $\text{Adv}^{\text{CDH}}(B)$ vogliamo dimostrare che $\text{Adv}^{\text{BDH}}(A) = \text{Adv}^{\text{CDH}}(B)$

¹Computational Diffie-Hellman

Capitolo 9

Firme Digitali

Una firma digitale è l'equivalente informatico di una tipica firma convenzionale. La struttura è molto simile a quella del Message Authentication, tuttavia qui abbiamo una struttura simmetrica in cui abbiamo una chiave segreta che serve a firmare e una chiave pubblica che serve per verificare. Una firma digitale deve garantire **non ripudiabilità** e questo è garantito dal fatto che la natura asimmetrica di questa procedura, fa sì che solo l'utente che possiede la chiave privata possa firmare. Uno schema di firma digitale è formato dai seguenti algoritmi:

- **Algoritmo di generazione della chiave**: che è un algoritmo randomizzato che restituisce una coppia (vk, sk) dove vk è la chiave di verifica, mentre sk è la chiave di firma.
- **Algoritmo di firma**: algoritmo che prende in input sk e un messaggio m e restituisce una firma σ o un simbolo di errore.
- **Algoritmo di verifica**: algoritmo deterministico che prende in input vk , m e σ e restituisce 1 se la firma è valida, 0 altrimenti.

Definiamo la sicurezza di uno schema di firma in maniera simile a quanto già fatto per quanto riguarda MA:

▪ DS: (KeyGen, Sign, VF)

```
EspDSuf-cma (A)
(vk, sk) ←R KeyGen(); (M, σ) ← ASig(sk, ·)(vk);
if (Ver(vk, M, σ) == 1) & (M ∈ Messaggi) &
    (A non aveva richiesto M all'oracolo)
    Return 1
else Return 0
```

$$\text{Adv}^{\text{uf-cma}}(A) = \Pr[\text{Esp}_{\text{DS}}^{\text{uf-cma}}(A) = 1]$$

In questo caso non abbiamo più bisogno di un oracolo di verifica come accadeva per il MAC poiché abbiamo che la chiave di verifica è pubblica.

9.1 Firma RSA

L'idea alla base di questo sistema di firma è quello di sfruttare la funzione trapdoor in modo inverso a quanto fatto precedentemente per cifrare. Nel caso di RSA:

- L'algoritmo di cifratura diventa l'algoritmo di verifica.
- L'algoritmo di decifratura diventa l'algoritmo di firma.
- L'algoritmo di generazione delle chiavi resta invariato.

Vediamo precisamente gli algoritmi:

CHIAVE PRIVATA

Sign_{N,d}(M) $M = Z_N^*$

If $M \notin Z_N^*$ return $\perp$; $\sigma \leftarrow M^d \bmod N$ ✓

Return σ

CHIAVE PUBBLICA

Ver_{N,e}(M, σ) ✓

If $(M \notin Z_N^* \vee \sigma \notin Z_N^*)$ return $\perp$

if $\sigma^e = M \bmod N$ Return 1

else Return 0

Purtroppo, a causa della natura di cifratura di RSA, se N è di 3000 bit, posso cifrare solo messaggi di 3000 bit e ciò lo rende poco usabile. Inoltre, lo schema non è sicuro poiché $Ver(1, 1)$ è sempre vero senza nemmeno dover chiedere mai nulla all'oracolo di firma. Inoltre, poiché e è pubblico¹, è possibile generare firme casuali ed elevarli a e per avere una coppia valida. E si può anche dimostrare l'esistenza di un altro attacco che permette la cifratura di messaggi arbitrari. Li vediamo in figura entrambi gli attaccanti rispettivamente come A e B.

A(μ, e): $\sigma \leftarrow_r Z_N^* \setminus \{1\}$ $M \leftarrow \sigma^e \bmod N$ (M, σ)	B(N, e) 1. $M \in m$ 2. $M_1, M_2 \in m$ $M_1 \cdot M_2 \neq M$ $M_1 \cdot M_2 = M \bmod N \implies M_1 = M \cdot M^{-1} \bmod N$ 3. $\text{Sign}(M_1) \rightarrow \sigma_1$ 4. $\text{Sign}(M_2) \rightarrow \sigma_2$ 5. output ($\sigma = \sigma_1 \cdot \sigma_2 \bmod N$)
---	--

9.2 Approccio Hash e Inverti

Ci serve un modo per far sì che M sia una stringa arbitraria e per "distruggere" le proprietà algebriche di RSA. Per fare ciò, ci basta utilizzare una funzione Hash come segue:

¹ e è pubblico poiché è la chiave di verifica

```

SignN,d(M)
  y ← HN(M);    // HN : {0, 1}* → ZN*
  σ ← yd mod N
  Return σ

```

```

VerN,e(M, σ)
  y ← HN(M);    y' = σe mod N;
  if (y = y') Return 1
  else Return 0

```

Dove H è resistente alle collisioni, intrattabile da invertire e che possa prendere messaggi arbitrari. Per verificare la sicurezza nel caso di Hash e inverti, ci mettiamo nuovamente nel contesto di Random Oracle, per cui:

```

EspDSuf-cma (A)
  ((N,e),(p,q,d)) ←R KeyGenRSA();
  H ←R Func({0,1}*, ZN*);
  (M,σ) ← AH(·), SignH,N,d(·) (N,e);
  if (VerN,e(M,σ) = 1) & (M ∈ Messaggi) &
    (A non aveva richiesto M all'oracolo)
    Return 1
  else Return 0

```

$$\text{Adv}^{\text{uf-cma}}(A) = \Pr[\text{Esp}_{\text{DS}}^{\text{uf-cma}}(A) = 1]$$

Come possiamo vedere, A adesso ha la necessità di avere un oracolo di Hash poiché l'hash è un random oracle.

Un teorema ci dice che: Sia A un avversario che fa al più q_s richieste di firma e q_h richieste all'oracolo di Hash. Esiste un avversario I che usa risorse comparabili ad A tale che:

$$\text{Adv}^{\text{uf-cma}}(A) \leq q_h \quad \text{Adv}^{\text{ow-rsa}}(I)$$

Dove q_h non è trascurabile. Per cui, questo fa' sì che abbiamo bisogno di un modulo RSA abbastanza grande da rendere q_h trascurabile. Tuttavia, maggiore è il modulo RSA, meno è scalabile, dunque, non potendo ignorare q_h , si passa a schemi randomizzati.

9.3 EC-DSA e Schorr's Signature

Vediamo in breve questo schema:

$$\begin{aligned}
 & \text{Keygen } q, G, g \\
 & x \leftarrow \{1, \dots, q\} \quad h \leftarrow g^x \\
 & H: \{0, 1\}^* \rightarrow \{1, \dots, q\} \\
 & v_K = (G, g, q, h, H) \quad M = \{0, 1\}^* \\
 & S_K = x
 \end{aligned}$$

$$\begin{aligned}
 & \text{Sign}_K(m) \\
 & k \leftarrow \{1, \dots, q\} \quad \text{SAREBBE UN PUNTO} \\
 & R \leftarrow g^k; \quad r \leftarrow R \bmod q \quad \text{QUINDI È UNA COSA} \\
 & \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \text{STANNA che dipende} \\
 & \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \text{dalla chiave} \\
 & s \leftarrow K^{-1} (H(m) + r x) \bmod q \\
 & \text{OUTPUT } (r, s)
 \end{aligned}$$

$$\begin{aligned}
 & \text{Ver}(v_K, m, (r, s)) \\
 & w \leftarrow s^{-1} \bmod q \\
 & w_1 \leftarrow g^{w H(m)} \\
 & w_2 \leftarrow h^{w r} \\
 & v \leftarrow w_1 \cdot w_2; \\
 & \text{IF } (v \bmod q == r) \text{ return 1} \\
 & \text{ELSE RETURN 0}
 \end{aligned}$$

Dimostriamo che funziona

$$v = u_1 \cdot u_2 = g^{w h(m)} \cdot h^{w x_2} = g^{w h(m)} \cdot g^{w x_2} = g^{w(h(m) + x_2)}$$

$$\text{Ma } w = s^{-1} \bmod q = [K^{-1}(h(m) + x_2)]^{-1} = K(h(m) + x_2)^{-1}$$

$$v = g^{K(h(m) + x_2)^{-1} \cdot (h(m) + x_2)} = g^K \rightarrow$$

$$\rightarrow v \bmod q = g^K \bmod q$$

Per quanto riguarda, invece, il sistema di **Schorr's Signature** abbiamo che:

keygen è come EC-SSA

$\text{Sign}_{v_K}(m)$:

$$z \leftarrow \{1, \dots, q\}$$

$$R \leftarrow g^z$$

$$c \leftarrow H(R, v_K, m)$$

$$z \leftarrow z + cx \bmod q$$

$$\text{RETURN } (R, z)$$

$\text{Ver}_{v_K}((R, z), m)$

$$c \leftarrow H(R, v_K, m)$$

$$\text{IF } (R \cdot h^c = g^z) \text{ output 1}$$

$$\text{ELSE output 0}$$

Dimostriamo che funziona

$$R \cdot h^c = g^z \quad h^c = g^z (g^x)^c = g^{z+cx} = g^z$$